Ministry of Education of the Republic of Belarus
Educational Institution
«Brest State Technical University»

Faculty of Electronic Information Systems
Department of Intelligent Information Technologies

EXPLANATORY NOTE

to the course project

«Development of an application for finding the optimal course of a sea vessel over open-sea points based on a genetic algorithm»

КР.ИИ2025.250978-01 81 00

Performed by: student of group ИИ-2025 _____
Li Haoran (Ли Хаожань)

Checked by: A. A. Kozinsky _____

Brest 2026

ABSTRACT

Explanatory note: 45 p., 17 fig., 14 tables, 31 sources, 2 appendices.

GENETIC ALGORITHM, SHIP COURSE, ROUTE PLANNING, NAVIGATION GRAPH, VISIBILITY TEST, WGS-84, FITNESS FUNCTION, DIJKSTRA ALGORITHM, A* ALGORITHM, PYQT5

The object of the work is the planning of the course of a sea vessel over the points of open sea. The aim is to develop a desktop application that finds the course with a genetic algorithm over the author's labelled dataset of WGS-84 points of four classes and compares it with Dijkstra's and A* algorithms.

The subject area and the methods of route search are analysed. A navigation graph is built on the water points; edges that cross land are removed by a visibility test based on the labelled points. The cost of a route is its length with penalties for coastal water and for the vicinity of land. The genetic algorithm uses a path encoding of variable length, a goal-directed random walk, tournament selection, crossover at a common node, re-routing and shortcut mutations and elitism.

The application «Sea Course GA» is written in Python with PyQt5, NetworkX, SciPy and Matplotlib. On seven test voyages the genetic algorithm found a navigable route in all 70 runs, in 57 of them within 1 % of the optimum (mean gap 0.90 %).

Educational Institution «Brest State Technical University»
Department of Intelligent Information Technologies

APPROVED
Head of the Department
_____ V. A. Golovko
«____» _____ 2026

ASSIGNMENT

for the course project

Student: **Li Haoran** (Ли Хаожань, 李浩然), group ИИ-2025

1. Topic of the project: **Development of an application for finding the optimal course of a sea vessel over open-sea points based on a genetic algorithm**.

2. Deadline for submission of the completed project: 12.10.2026.

3. Initial data for the project.

General requirements for the application:

The aim of the work is to create a cross-platform desktop application for finding the optimal course of a sea vessel over open-sea points using a genetic algorithm, with a comparison against classical path-finding algorithms.

– the application must run under Windows 10/11 and Linux (Ubuntu 20.04+, Fedora 38+);

– the application must use the student's own dataset: 150 000 points in the WGS-84 format divided into 4 classes;

– the user interface must be graphical (GUI) and intuitive;

– the development tools are chosen by the developer (recommended: Python + NetworkX + PyQt/Tkinter + Matplotlib/folium);

– the application must allow the GA parameters to be set: population size, number of generations, crossover probability, mutation probability, penalties for dangerous zones;

– the results must be saved to local files (route, convergence plots, parameter log).

Functional requirements for the application:

– loading of the dataset from a file (WGS-84, 4 classes of points);

– construction of a graph on the points (by distance or by visibility);

– implementation of a genetic algorithm for finding the course of the vessel;

– consideration of constraints: avoidance of dangerous zones (classes of points), minimisation of the path length;

– implementation or reuse of Dijkstra's and A* algorithms for comparison (already implemented by the student);

– visualisation: map with the points, the course of the vessel (GA), the routes of Dijkstra and A*;

– plotting of the convergence graph of the GA;

– export of the results: map images, routes, parameter log.

Dataset: labelled dataset (150 000 WGS-84 points, 4 classes).

4. Content of the explanatory note:

Introduction

1 System analysis of the subject area and statement of the problem of designing an application for finding a sea vessel's course

1.1 Analysis of the subject area: maritime navigation, course plotting, consideration of dangerous zones

1.2 Review of methods: classical path-finding algorithms (Dijkstra, A*) and genetic algorithms

1.3 Justification of the choice of the genetic algorithm and tools

2 Design of the application for finding a sea vessel's course

2.1 Development of the application architecture (loading module, graph construction module, GA module, visualisation module)

2.2 Design of the user interface

2.3 Formalisation of the problem: chromosome (route), fitness function (length + penalties), operators

3 Implementation and testing of the application for finding a sea vessel's course

3.1 Implementation of the module for loading and processing the WGS-84 dataset

3.2 Implementation of the graph construction module

3.3 Implementation of the genetic algorithm with constraints
3.4 Testing, comparison with Dijkstra and A*, analysis of the results
Conclusion. List of abbreviations. References
Appendix A. User guide. Appendix B. Listings of the key modules

5. List of graphic material:
   1) Statement of the problem of creating an application for finding a sea vessel's course (poster – A4)
   2) Architecture of the application for finding a sea vessel's course (poster – A4)
   3) Formalisation of the problem: chromosome and fitness function (poster – A4)
   4) Structural diagram of the application (drawing – A4)
   5) Test results of the application (map with the courses, convergence plots, comparison tables) (drawing – A4)
6. Consultants on the project: –
7. Date of issue of the assignment: 25.09.2026.
8. Calendar schedule of the work on the project:

Part of the project	**Dates**	**Share of work**
Section 1	25.09.2026 – 02.10.2026	20 %
Section 2	03.10.2026 – 07.10.2026	45 %
Section 3	07.10.2026 – 09.10.2026	15 %
Conclusion	10.10.2026	10 %
Formatting of the project	11.10.2026	10 %

Supervisor _____ A. A. Kozinsky

The assignment was accepted for execution on 25.09.2026

Student _____ Li Haoran

CONTENTS

INTRODUCTION.....5
1 System analysis of the subject area and statement of the problem of designing an application for finding a sea vessel's course.....6
1.1 Analysis of the subject area: maritime navigation, course plotting, consideration of dangerous zones.....6
1.2 Review of methods: classical path-finding algorithms (Dijkstra, A*) and genetic algorithms.....8
1.3 Justification of the choice of the genetic algorithm and tools.....10
2 Design of the application for finding a sea vessel's course.....12
2.1 Development of the application architecture.....12
2.2 Design of the user interface.....14
2.3 Formalisation of the problem: chromosome, fitness function, operators.....17
3 Implementation and testing of the application for finding a sea vessel's course.....23
3.1 Implementation of the module for loading and processing the WGS-84 dataset.....23
3.2 Implementation of the graph construction module.....24
3.3 Implementation of the genetic algorithm with constraints.....27
3.4 Testing, comparison with Dijkstra and A*, analysis of the results.....28
CONCLUSION.....35
LIST OF ABBREVIATIONS.....36
REFERENCES.....37
APPENDIX A User guide.....40
APPENDIX B Listings of the key modules (program text).....46

INTRODUCTION

Planning the route of a ship is one of the routine but responsible tasks of navigation. Before every voyage the navigating officer prepares a passage plan: a sequence of waypoints from the berth of departure to the port of destination that keeps the vessel in water of sufficient depth and at a safe distance from the shore [1, 2]. The shortest line between two points on the Earth, the arc of a great circle, is rarely acceptable for this purpose: near the coast it crosses peninsulas, islands and shallow banks, so the real course is a compromise between the length of the passage and its safety.

Computer route planning reduces this compromise to the search for a path of minimal cost on a model of the sea area. The classical exact methods, Dijkstra's algorithm and the A* algorithm, solve the problem optimally when the cost of a route is the sum of the costs of its legs [3, 4]. Evolutionary methods, and genetic algorithms in particular, do not guarantee optimality but place no restrictions on the form of the criterion and return a whole population of alternative routes [5, 6]. For a practical application it is therefore useful to have both: the exact search gives the reference value, the genetic algorithm gives a flexible search whose quality can be measured against that reference.

The object of the course project is the planning of the course of a sea vessel over the points of open sea. The subject is the genetic algorithm for the search of the course on a graph built on labelled geographic points and its comparison with Dijkstra's and A* algorithms.

The aim of the project is to develop a cross-platform desktop application that finds the course of a vessel over the open-sea points of the author's own dataset (WGS-84, four classes of points) with a genetic algorithm, takes dangerous zones into account and compares the result with the exact searches.

To reach the aim, the following tasks are solved:

– to analyse the subject area and the methods of route search, and to justify the choice of the genetic algorithm and of the development tools;

– to design the architecture of the application and its user interface;

– to formalise the problem: the navigation graph, the fitness function with penalties for dangerous zones, the encoding of a route as a chromosome and the genetic operators;

– to implement the modules of loading the dataset, building the graph, the genetic algorithm, the exact searches, visualisation and export;

– to test the application and to compare the genetic algorithm with Dijkstra's and A* algorithms on a set of voyages in different sea areas.

The application received the short name «Sea Course GA». It is written in Python with the libraries NetworkX, NumPy, SciPy, Matplotlib and PyQt5 and runs under Windows and Linux.

1 System analysis of the subject area and statement of the problem of designing an application for finding a sea vessel's course

1.1 Analysis of the subject area: maritime navigation, course plotting, consideration of dangerous zones

In the practice of navigation the route of a vessel is prepared in advance as a passage plan. The IMO guidelines for voyage planning [1] divide the work into four stages: appraisal of all information relevant to the voyage, planning of the route, execution of the plan and monitoring of the progress of the vessel. At the planning stage the route is laid on the chart from berth to berth as a sequence of waypoints connected by straight legs, and the planned track has to keep clear of dangers to navigation with a sufficient margin of safety.

Two kinds of lines are used for the legs. A rhumb line (loxodrome) crosses all meridians at the same angle and is convenient to steer, while the arc of a great circle (orthodrome) is the shortest line between two points on the sphere [2]. On the legs of a coastal passage, which are tens of kilometres long, the difference between the two is negligible, and in this work every leg is measured as an arc of a great circle.

Coordinates of waypoints are given as geodetic latitude φ and longitude λ in the WGS-84 system. The reference ellipsoid of WGS-84 has the semi-major axis a = 6 378 137 m and the inverse flattening 1/f = 298.257223563 [7]. For the comparison of route variants the Earth is replaced by a sphere of the mean radius R = (2a + b)/3 = 6 371.0088 km, where b is the semi-minor axis. The distance between two points is then computed with the haversine formula [8]:

(1.1)

where d – great-circle distance between the points, km;

R – mean radius of the Earth, R = 6 371.0088 km;

φ_1, φ_2 – latitudes of the points, rad;

$\Delta\varphi$, $\Delta\lambda$ – differences of latitude and longitude of the points, rad.

The error of the spherical model with respect to the ellipsoid does not exceed about 0.5 %, which is acceptable when routes are compared with one another and the same model is used for all of them.

The main dangers on a coastal passage are the land itself and the shallow water close to the shore. In the course project they are described not by a vector chart but by the author's own labelled dataset of geographic points. Each point carries its WGS-84 coordinates and one of four classes; the classes and the role of each of them in the route search are listed in table 1.1.

Table 1.1 – Classes of points of the dataset

Class code	Meaning	Points in the file	Points after cleaning	Role in the search
`OPEN_SEA`	deep water far from the shore	55 951	54 701	graph node, no penalty
`COASTAL_SEA`	navigable water near the shore	38 092	36 258	graph node, penalised
`NEAR_COAST`	land along the shoreline	30 952	29 271	obstacle
`COASTLINE`	land remote from the sea	27 843	27 533	obstacle
Total	–	152 838	147 763	–

The file `merged.json` is a GeoJSON collection of 152 838 point features collected in 28 working sessions between April and July 2026. Besides the coordinates and the class, every feature stores the time of collection, the elevation, the relief value `noaa_depth` (negative below sea level, positive above it), the type of that value and nine counters of OpenStreetMap objects near the point (piers, lighthouses, breakwaters, harbours, rivers, lakes, beaches, cliffs, industrial objects). For the route search only the coordinates, the class and the relief value are used: the last one serves to check the labels.

The check is needed because a part of the points carries a class that contradicts the measured relief. For example, 1 834 points of the class `COASTAL_SEA` lie more than 5 m above sea level, and 1 681 points of the class `NEAR_COAST` lie more than 5 m below it. In total 5 075 points (3.32 %) are removed by this rule; after cleaning the median relief value is −2 173 m for open sea, −28 m for coastal sea, +11 m for coastal land and +210 m for inland points, which agrees with the meaning of the classes.

The points cover ten sea areas; their location is shown in figure 1.1. The density of the points is not uniform: the water near the coast and the straits is sampled densely, while the open sea is represented by scattered points several kilometres apart. This feature of the data determines the way the navigation graph is built (subsection 2.3).

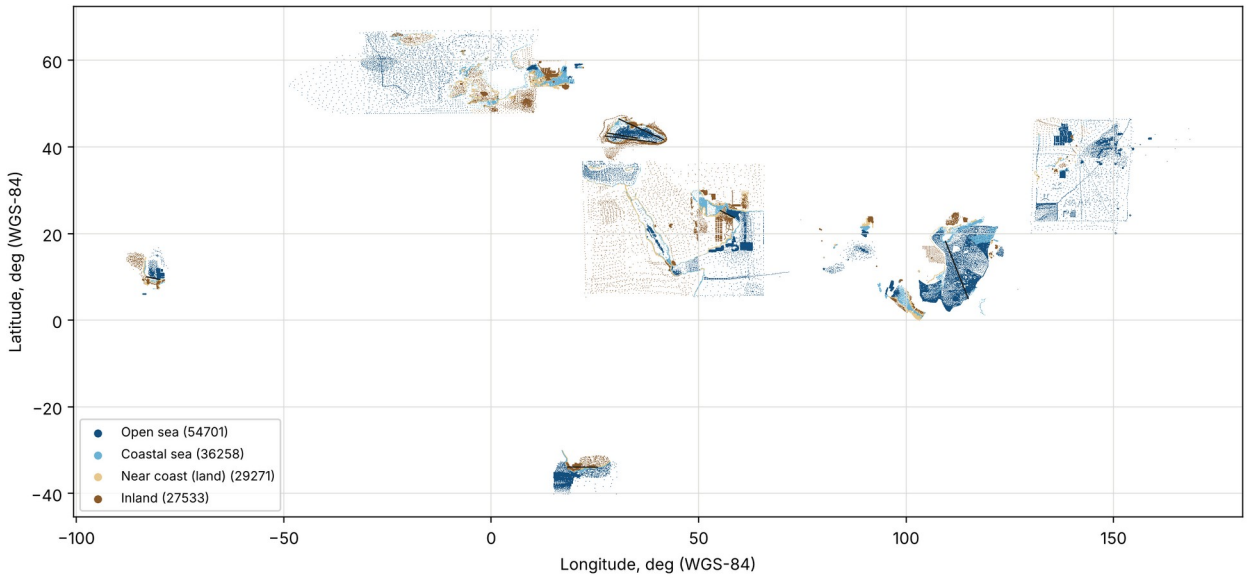


Figure 1.1 – Points of the dataset after cleaning and the test voyages (black lines)

Thus, in terms of the subject area the task of the application is the following: for the given departure and destination positions, find a sequence of water points of the dataset that can be sailed leg by leg without crossing land, that is as short as possible and that avoids coastal water and the vicinity of land as far as the length allows.

1.2 Review of methods: classical path-finding algorithms (Dijkstra, A*) and genetic algorithms

Methods of ship route planning are reviewed in [9–11]. They differ in the model of the sea area and in the criterion. The area is represented either by a regular grid of cells, or by a graph whose nodes are chosen points of the water, or by a continuous space in which a route is a polyline with free vertices. The criterion may be the distance, the time, the fuel consumption or the risk; in weather routing it depends on the forecast of wind and waves [10, 11]. In the course project the model is a graph on the labelled points and the criterion is the length of the route with penalties for dangerous zones.

Dijkstra's algorithm [3] finds the cheapest paths from a source node to all nodes of a graph with non-negative edge costs. The nodes are taken from a priority queue in the order of increasing distance from the source; when a node is taken, its distance is final and its edges are relaxed. With a binary heap the running time is $O((|V| + |E|) \log |V|)$, where $|V|$ and $|E|$ are the numbers of nodes and edges [12]. If only one target is needed, the search stops when the target is taken from the queue.

The A* algorithm [4] orders the queue by the estimate $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the best known path from the source to n and $h(n)$ is a heuristic estimate of the

remaining cost. If h never overestimates the remaining cost (an admissible heuristic) the path found is optimal; if, in addition, $h(u) \le c(u, v) + h(v)$ for every edge (a consistent heuristic), no node has to be expanded twice [13]. For a route at sea the natural heuristic is the great-circle distance to the destination. A* is widely used in vessel path planning; for example, in [14] it is extended with risk models of obstacles, currents and traffic separation.

Both algorithms need a graph. On a regular grid the moves are limited to the neighbouring cells, which gives zigzag routes, and the size of the problem grows with the square of the resolution. In a visibility graph [15] two vertices are joined only if the straight segment between them does not cross an obstacle; the edges are then legs that can actually be sailed. Other exact methods, such as dynamic programming [16], are applied when the cost of a leg depends on time or speed.

Genetic algorithms were proposed by J. Holland [5] and became a general optimisation method after the works of D. Goldberg [6]; modern presentations are given in [17–19]. A genetic algorithm keeps a population of candidate solutions encoded as chromosomes. In every generation parents are selected according to their fitness, recombined by a crossover operator and changed by a mutation operator, and the offspring form the next population. Tournament selection, in which the best of k randomly chosen individuals becomes a parent, is simple and controls the selection pressure by k [20]. Constraints are taken into account either by penalty terms in the fitness function, or by repair of infeasible individuals, or by an encoding in which every chromosome is feasible [21].

For the shortest path problem C. W. Ahn and R. S. Ramakrishna proposed an encoding in which a chromosome is the sequence of nodes of a path of variable length, the crossover exchanges the parts of two parents at a common node and the mutation replaces a part of the path by a new random one [22]. With this encoding every individual is a feasible path, so no penalty for broken routes is needed; the same work derives the population size required for a reliable search. In the maritime domain evolutionary algorithms were applied to ship trajectories in collision situations [23] and to multi-objective weather routing [24].

The properties of the three approaches are compared in table 1.2.

Table 1.2 – Comparison of the route search methods

Property	Dijkstra	A*	Genetic algorithm
Result	optimal path	optimal path	good path, optimality not guaranteed
Requirements to the criterion	additive, edge costs ≥ 0	additive, admissible heuristic	any computable function of the route
Nodes examined	all nodes cheaper than the target	fewer than Dijkstra	only the nodes of the routes of the population
Alternatives	one path	one path	a population of routes
Running time	low	lower than Dijkstra	high, set by N and G_{max}
Main parameters	–	heuristic	N, G_{max}, p_c, p_m, operators

1.3 Justification of the choice of the genetic algorithm and tools

When the cost of a route is a sum of edge costs, the exact algorithms are faster than any evolutionary method, and the course project does not claim otherwise. The genetic algorithm is chosen as the main search method for other reasons:

– the fitness function of the genetic algorithm can be extended with terms that are not sums over edges (the number of course changes, the largest leg, a limit on the time in coastal water) without changing the algorithm, whereas Dijkstra's and A* algorithms would then lose their optimality;

– the algorithm returns a population of different routes, for example passing inshore or offshore of a cape, from which the navigator can choose;

– the search can be stopped at any generation and still gives a valid route;

– on the same graph and with the same cost model the exact optimum is known, so the quality of the genetic algorithm can be measured objectively as the relative excess of its cost over the optimum.

The encoding of a route as a path of variable length after Ahn and Ramakrishna [22] is taken as the basis, because with it every chromosome is a sequence of legs that exist in the graph. Since the graph contains only edges that do not cross land, the main constraint of the problem is satisfied by construction, and the other dangers are taken into account by penalties in the fitness function [21]. Dijkstra's and A* algorithms written by the author earlier for a regular grid are reused on the point graph and serve as the reference.

The development tools are chosen according to the recommendation of the assignment (table 1.3).

Table 1.3 – Development tools

Tool	Purpose in the application
Python 3.10 or newer	programming language; the same code runs under Windows and Linux
NetworkX [25]	storage of the navigation graph, connected components
NumPy [26]	vectorised geometry: coordinates, distances, edge attributes
SciPy [27]	k-d tree (cKDTree) for nearest-neighbour queries
PyQt5 [28]	graphical user interface, background threads
Matplotlib [29]	map and convergence plots embedded in the window, PNG export
folium	optional interactive HTML map in the export
pandas, ijson	reading of GeoJSON, CSV and JSON files
pytest	automated tests

A k-d tree [30] answers the query «which points lie near this one» in logarithmic time on average and is used both to build the edges of the graph and to test whether a leg crosses land. PyQt5 was preferred to Tkinter because of the ready-made widgets for tables, tabs and

progress indication, the thread classes with signals and the direct embedding of Matplotlib figures.

Statement of the problem. It is required to develop the desktop application «Sea Course GA» which:

1) reads the labelled dataset (GeoJSON, CSV or JSON, WGS-84, four classes of points) and removes the points whose class contradicts the relief value;

2) builds a navigation graph on the water points of an area around the voyage, joining points by distance or by visibility;

3) computes the cost of every edge as its length increased by penalties for coastal water and for the vicinity of land, with weights set by the user;

4) finds the course with a genetic algorithm whose population size, number of generations, crossover and mutation probabilities, tournament size, elitism, stagnation limit and random seed are set in the window;

5) finds the optimal route on the same graph with Dijkstra's and A* algorithms and compares the three results;

6) shows the points, the graph and the routes on a map, draws the convergence curve of the genetic algorithm and saves the map, the routes, the comparison table and the parameter log to files;

7) runs under Windows 10/11 and Linux (Ubuntu 20.04+, Fedora 38+) and keeps the window responsive during long computations.

2 Design of the application for finding a sea vessel's course

2.1 Development of the application architecture

The application is divided into three layers (figure 2.1). The presentation layer contains the main window and the class of background threads. The application layer is a single class, `CourseSession`, which keeps the state of one working session: the loaded points, the current graph, the departure and destination, the penalty weights and the results of the last runs. The domain layer is the package `maritime_route.pointgraph`; its modules do not depend on Qt and can be used from scripts, which is how the experiments of subsection 3.4 were run.

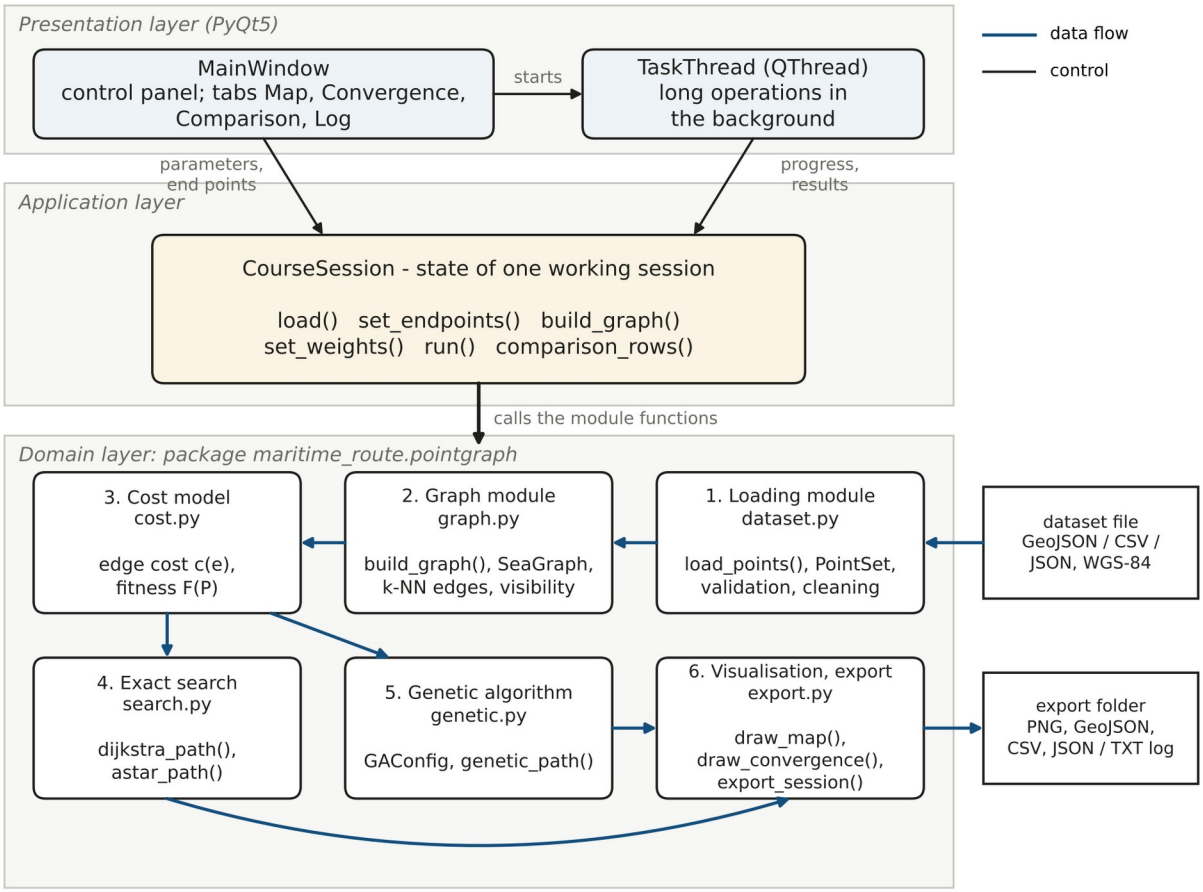


Figure 2.1 – Architecture of the application

The assignment names four modules: loading, graph construction, the genetic algorithm and visualisation. In the implementation the cost model and the exact searches were moved to modules of their own, because they are shared by the genetic algorithm and by Dijkstra's and A* algorithms. The modules and their size are listed in table 2.1.

Table 2.1 – Modules of the application

Module (file)	Responsibility	Lines
Loading (`dataset.py`)	reading the file, validation of coordinates, removal of class/relief conflicts, class `PointSet`	214
Graph (`graph.py`)	selection of the area, k nearest neighbours, edge attributes, visibility test, components, class `SeaGraph`	407
Cost model (`cost.py`)	edge cost c(e), terms of the fitness function F(P)	100
Exact search (`search.py`)	Dijkstra's and A* algorithms, class `PathResult`	213
Genetic algorithm (`genetic.py`)	encoding, initial population, operators, elitism, stopping rules, class `GAConfig`	437
Session (`session.py`)	state of the session, voyage presets, comparison table	234
Visualisation and export (`export.py`)	map and convergence drawings, export of PNG, GeoJSON, CSV, JSON, TXT and HTML files	300
Window (`main_window.py`)	widgets, event handlers, start of background tasks	629
Threads (`workers.py`), start (`app.py`)	class `TaskThread`, command line, creation of the window	91

Three helper modules are taken from the author's diploma code base without changes: the file reader `ais_loader.py` (GeoJSON, CSV and JSON with automatic recognition of the column names), the label check `cleaning.py` and the geodesy functions `geodesy.py` (haversine distance). The neural network and the web service of the diploma project are not used by the course project.

The data pass through the modules in one direction. The loading module turns the file into a `PointSet` of parallel NumPy arrays (latitude, longitude, class, sea region). The graph module selects the water points of the area and returns a `SeaGraph` with a NetworkX graph whose edges carry three attributes: the length, the share of coastal water and the distance to land. The cost model writes the attribute `weight` on every edge; because the attributes are stored, a change of the penalty weights only recomputes the weights and does not rebuild the graph. The three search algorithms read the same `weight` and return a `PathResult`. The visualisation module draws the session and saves the results.

The main classes and their relations are shown in figure 2.2. All parameter sets (`GraphConfig`, `CostWeights`, `GAConfig`) are immutable data classes; they are written to the parameter log together with the results, so that every run can be repeated.

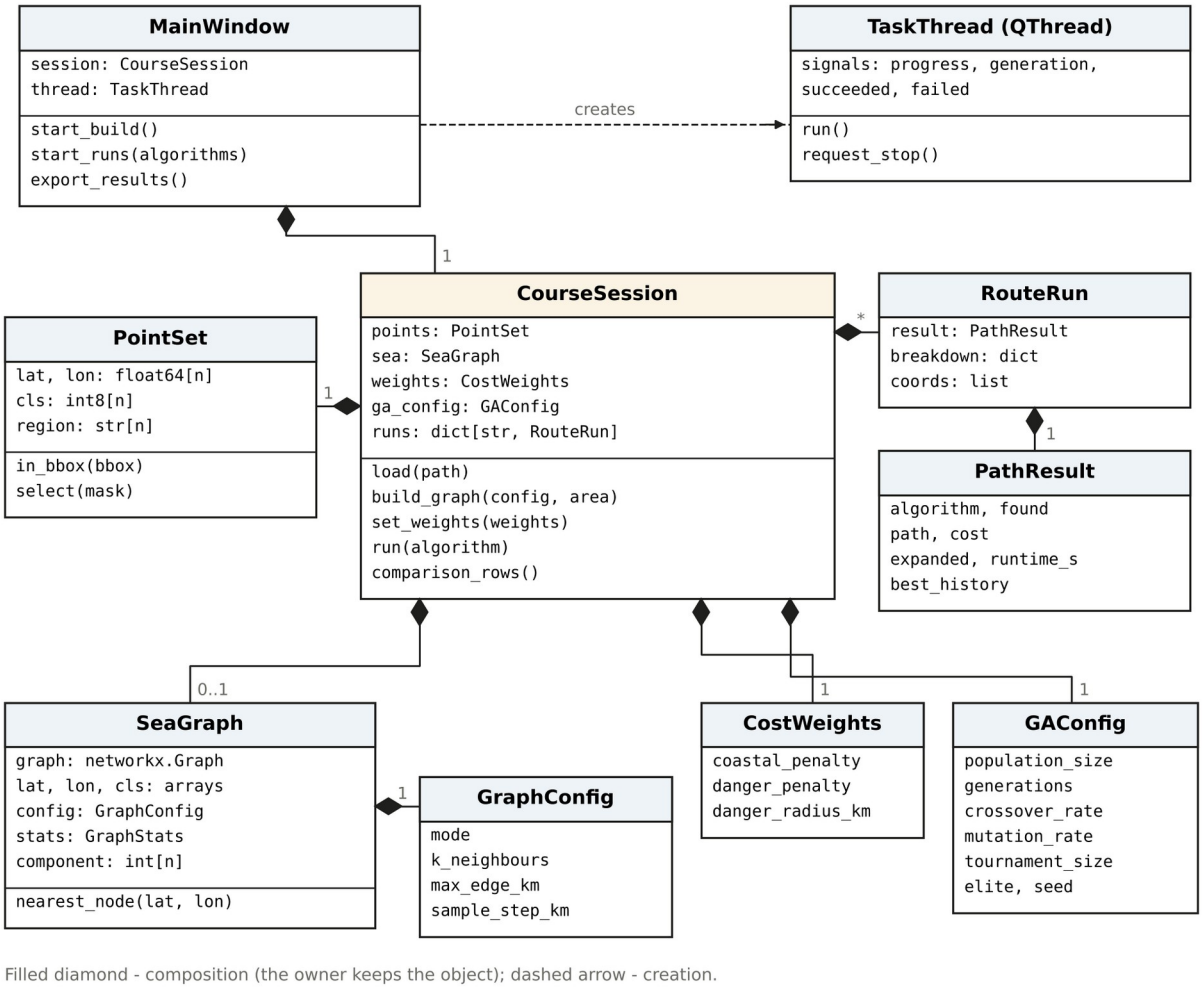


Figure 2.2 – Main classes of the application

Loading of the dataset takes a few seconds, the construction of a graph up to several seconds and a run of the genetic algorithm several seconds more. If these operations were executed in the thread of the user interface, the window would not respond to the user. Therefore every long operation is executed in a `TaskThread` (a subclass of `QThread`). The thread reports progress through the signals `progress(str, float)` and `generation(int, float, float)`, returns the result through `succeeded(object)` and an error through `failed(str)`. Qt delivers the signals to the thread of the window, so the widgets are changed only there. While a task is running, all widgets that change the session are disabled, and the button «Stop» asks the genetic algorithm to finish after the current generation.

2.2 Design of the user interface

The main window (figure 2.3) consists of a scrollable control panel on the left, a set of four tabs on the right, a menu and a status bar with a progress indicator. The groups of the

control panel are numbered in the order in which the user normally works: the dataset is opened, the voyage is set, the graph is built, the penalties and the parameters of the genetic algorithm are adjusted, and the algorithms are run.

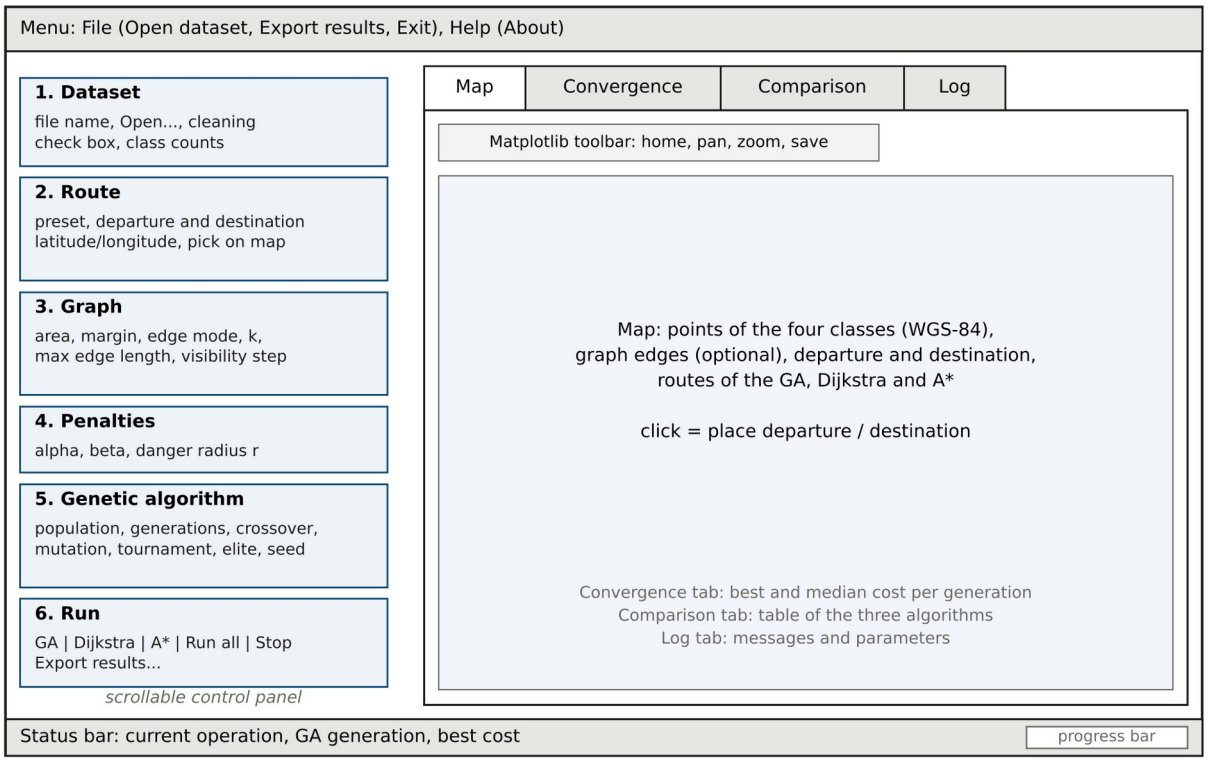


Figure 2.3 – Layout of the main window

The groups of the control panel have the following content:

1) «Dataset» – the name of the opened file, the button «Open...», the check box that switches the removal of class/relief conflicts and the number of points of every class;

2) «Route» – the list of preset voyages, the latitude and longitude of the departure and the destination in decimal degrees (four digits after the point) and two buttons that let the user pick the positions with the mouse on the map;

3) «Graph» – the area (around the route with a margin in degrees, or one of the ten sea regions of the dataset), the edge mode (by visibility or by distance), the number of nearest neighbours k, the maximal edge length and the step of the visibility test;

4) «Penalties» – the coastal water penalty α, the danger zone penalty β and the danger zone radius r;

5) «Genetic algorithm» – population size, number of generations, crossover and mutation probabilities, tournament size, number of elite individuals, stagnation limit and random seed;

6) «Run» – buttons that run the genetic algorithm, Dijkstra's or A* algorithm alone or all three in a row, the button «Stop» and the button «Export results...».

The tab «Map» shows the points of the four classes in different colours, optionally the edges of the graph, the departure and destination markers and the routes of the three algorithms; the Matplotlib toolbar gives zoom, pan and saving of the picture. The tab «Convergence» shows the cost of the best route and the median cost of the population in every generation. The tab «Comparison» contains a table with one row per algorithm: the cost, the length, the length in coastal water, the danger-weighted length, the closest approach to land, the search effort, the running time and the excess of the cost over the optimum. The tab «Log» collects all messages of the session.

The ranges of the input fields are given in table 2.2. They are enforced by the spin boxes themselves, so the user cannot enter a value that the algorithms do not accept.

Table 2.2 – Parameters of the user interface

Parameter	Range	Default
Latitude / longitude of the end points, deg	−90…90 / −180…180	preset
Margin around the route, deg	0.5…10	2.0
Nearest neighbours k	2…30	10
Maximal edge length, km	2…200	40
Step of the visibility test, km	0.2…10	1.0
Coastal water penalty α	0…10	0.5
Danger zone penalty β	0…20	2.0
Danger zone radius r, km	0.5…30	10
Population size N	4…1000	100
Number of generations G_{max}	1…5000	300
Crossover probability p_c	0…1	0.80
Mutation probability p_m	0…1	0.30
Tournament size k_t	2…20	3
Elite individuals e	0…50	2
Stagnation limit, generations (0 – off)	0…5000	80
Random seed	0…999999	42

The upper limit of the danger zone radius equals the radius within which the graph module looks for land near an edge (30 km); a larger radius would make the penalty depend on land that was not examined. Errors (an unreadable file, an area without water points, departure and destination in different parts of the graph) are shown in a message box and repeated in the log.

2.3 Formalisation of the problem: chromosome, fitness function, operators

Navigation graph. Let the departure s_0 and the destination t_0 be given, and let A be a rectangle of latitude and longitude that contains both of them with a margin. The nodes of the graph G = (V, E) are the points of the classes `OPEN_SEA` and `COASTAL_SEA` that lie in A. Each node is joined with its k nearest water nodes that are not farther than l_{max}. In the mode «by distance» all such edges are kept. In the mode «by visibility» an edge is kept only if the straight segment between its end nodes does not cross land; the test is described in subsection 3.2. The departure and destination are attached to the nearest nodes s and t of the graph.

Every edge e = (u, v) has three attributes: its great-circle length l(e) by formula (1.1); the share of coastal water $s(e) \in \{0; 0.5; 1\}$, the mean of the coastal flags of its end nodes; and the distance $d_{land}(e)$ from the segment to the nearest land point.

Cost of an edge. The cost of an edge is its length increased by two penalties:

(2.1)

where $c(e)$ – cost of the edge, weighted km;

$l(e)$ – great-circle length of the edge, km;

α – penalty for sailing in coastal water;

$s(e)$ – share of the edge in coastal water;

β – penalty for sailing close to land;

$\varphi(e)$ – danger factor of the edge.

The danger factor equals one for an edge that touches land and decreases linearly to zero at the border of the danger zone of radius r:

(2.2)

where $d_{land}(e)$ – distance from the edge to the nearest land point, km;

r – radius of the danger zone, km.

Because $\alpha \geq 0$, $\beta \geq 0$ and the factors s(e) and $\varphi(e)$ are not negative, $c(e) \geq l(e)$ for every edge. This property is used below to prove that the heuristic of A* is admissible.

Fitness function. A route is a simple path $P = (v_0 = s, v_1, \ldots, v_m = t)$ in G. Its cost, which is the fitness of the genetic algorithm (to be minimised), is the sum of the costs of its edges and splits into three terms:

(2.3)

where $F(P)$ – cost of the route, weighted km;

m – number of legs of the route;

$L(P)$ – length of the route, km;

$L_c(P)$ – length sailed in coastal water, the sum of $l(e)\cdot s(e)$, km;

$L_d(P)$ – danger-weighted length, the sum of $l(e)\cdot \varphi(e)$, km.

The problem is to find the route P^* with the minimal cost $F(P^*) = F^*$. The constraint «the route does not cross land» is satisfied by the construction of the graph in the visibility mode, so the fitness function contains only the length and the penalties. The terms L, L_c and L_d and the closest approach of the route to land are reported separately, so the user can see what the route pays for.

Exact searches. Dijkstra's algorithm minimises the same sum of edge costs. A* uses the estimate

(2.4)

where $g(n)$ – cost of the best known path from s to the node n;

$h(n)$ – great-circle distance from n to the destination node t by formula (1.1).

The length of any path between two nodes is not smaller than the great-circle distance between them, and the cost of an edge is not smaller than its length (formula (2.1)). Hence h(n) never exceeds the cost of the remaining path, and $h(u) \le l(u, v) + h(v) \le c(u, v) + h(v)$ for every edge by the triangle inequality on the sphere. The heuristic is admissible and consistent, so A* returns the same optimal cost as Dijkstra's algorithm.

Chromosome. Following [22], a chromosome is the route itself: the sequence of node numbers from s to t (figure 2.4, a). Every two neighbouring genes are joined by an edge of G and no node occurs twice. The length of the chromosome is variable: a route along a winding coast may contain two hundred nodes, a route across the open sea a few dozen.

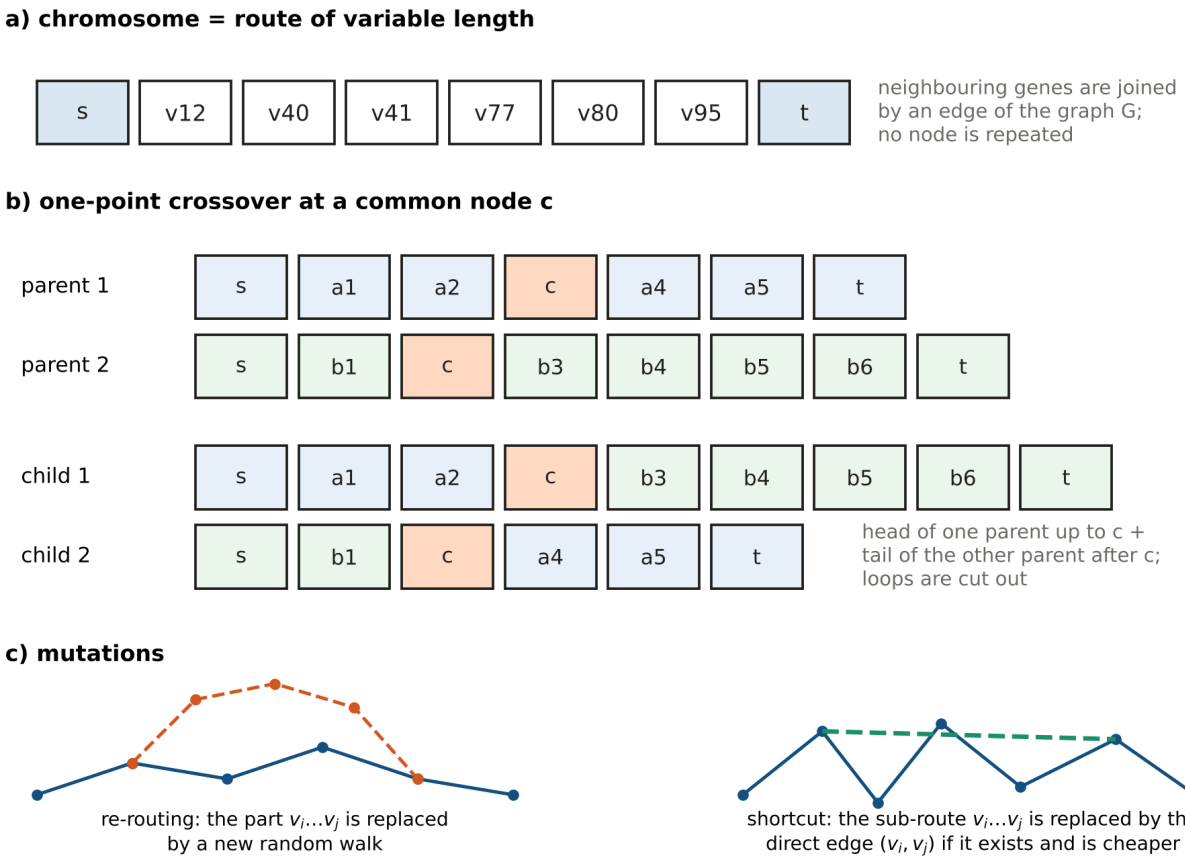


Figure 2.4 – Encoding of a route and the genetic operators

Initial population. Each initial route is grown by a randomised depth-first walk from s to t. At a node u the walk chooses the next node v among the unvisited neighbours with the probability

(2.5)

where κ – coefficient of attraction to the goal, drawn for every walk from the interval [1; 15];

$\gamma(u, v)$ – progress towards the goal per unit of cost of the edge (u, v), $-1 \le \gamma \le 1$;

w – runs over the unvisited neighbours of u.

A small κ gives a meandering route, a large κ an almost straight one, so the population starts with different shapes. If the walk reaches a node without unvisited neighbours it steps back (backtracking). In half of the cases the walk is first led to a random intermediate node taken in a circle around the midpoint between its end nodes; the radius of the circle is 0.1–0.5 of the distance between them. This spreads the population over different corridors, for example inshore and offshore of a cape.

Selection is a tournament of $k_t = 3$ individuals [20]: the route with the lowest cost among three randomly chosen routes becomes a parent.

Crossover is applied to a pair of parents with the probability p_c. A node c that both parents pass through (except s and t) is chosen at random; the first child is the head of the first

parent up to c followed by the tail of the second parent after c, and the second child is formed symmetrically (figure 2.4, b). Both children are valid routes, because the edges at the junction belong to the parents. If a node appears twice, the loop between its occurrences is cut out. Parents without a common node are copied unchanged.

Mutation is applied to each child with the probability p_m and is one of two moves (figure 2.4, c). The re-routing (60 % of mutations) chooses two positions of the route that are 2–60 genes apart and replaces the part between them by a new random walk that avoids the other nodes of the route. The shortcut (40 %) replaces a sub-route $v_i \dots v_j$ by the direct edge (v_i, v_j) if such an edge exists and is cheaper.

Elitism and stopping. The e best routes pass to the next generation unchanged, so the best cost never increases. The algorithm stops after G_{max} generations, after a given number of generations without improvement, or when the user presses «Stop». The result is the best route found together with the curves of the best and median cost per generation.

The flowchart of the algorithm is shown in figure 2.5. All random numbers are taken from one generator initialised with the seed from the window, so a run with the same parameters gives the same route.

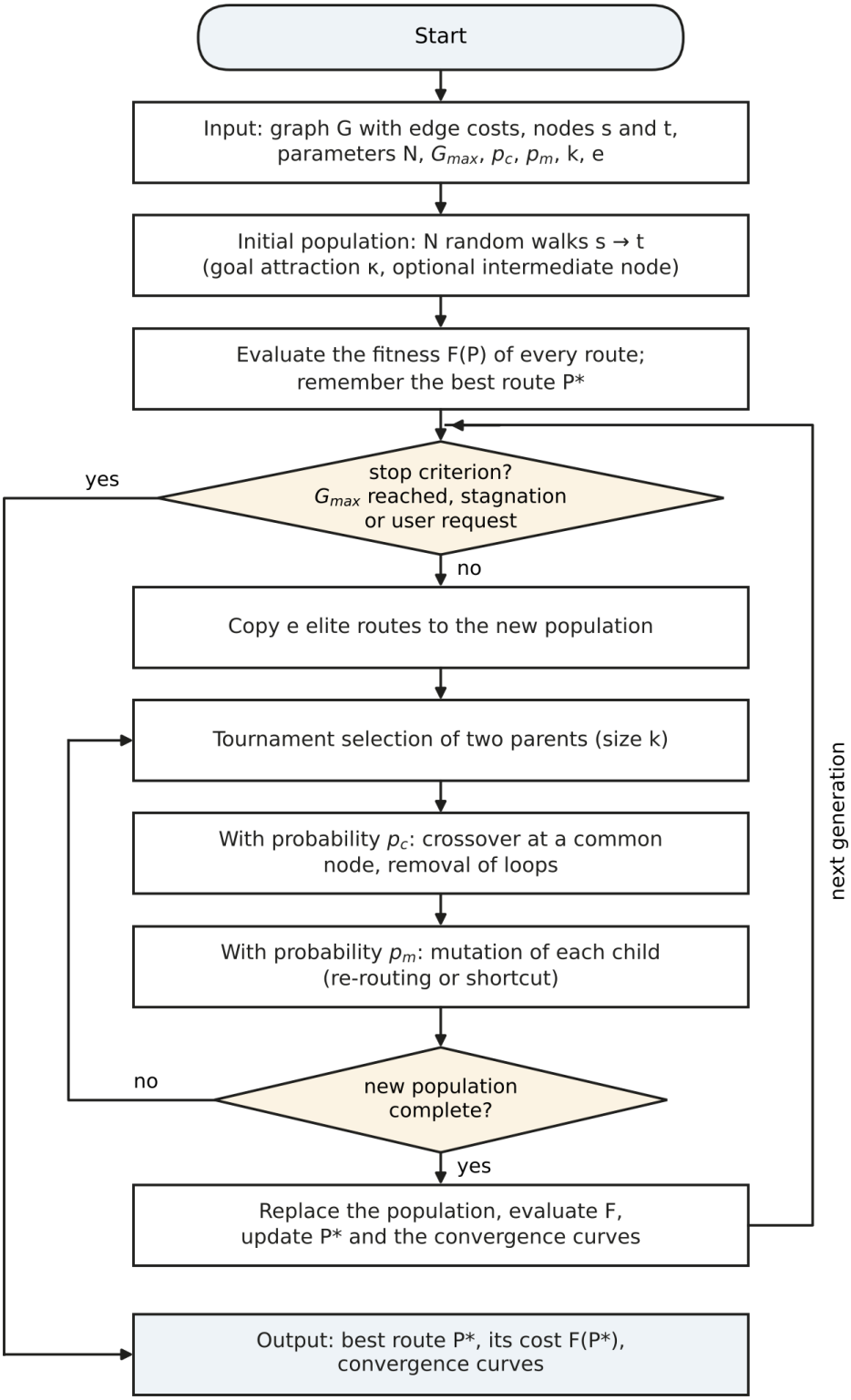


Figure 2.5 – Flowchart of the genetic algorithm

The quality of a route found by the genetic algorithm is measured by its relative excess over the optimum found by Dijkstra's or A* algorithm on the same graph:

(2.6)

where δ – gap between the cost of the GA route and the optimum, %;

$F(P_{GA})$ – cost of the route found by the genetic algorithm;

F^* – optimal cost.

3 Implementation and testing of the application for finding a sea vessel's course

3.1 Implementation of the module for loading and processing the WGS-84 dataset

The function `load_points(path, clean=True)` of the module `dataset.py` reads a file and returns a pair: the `PointSet` and a `LoadSummary` with the counts of every processing step. The file may be a GeoJSON collection of points (extension `.geojson` or `.json` with the key `features`), a CSV table or a JSON array of records. Every record needs a latitude, a longitude and the attribute `class_code`; for GeoJSON the coordinates of the point geometry take precedence over the attributes, because the geometry is the authoritative position in that format.

Processing consists of four steps:

1) the file is read by `load_any`; usual alternative column names (for example `lat`/`lon`) are mapped to `latitude`/`longitude`;

2) `validate` drops rows with missing coordinates or coordinates outside the ranges −90…90° and −180…180°;

3) rows whose `class_code` is not one of the four known codes are dropped;

4) if the check box «Remove class/depth conflicts» is on, the rule of subsection 1.1 is applied: a water point lying more than 5 m above sea level or a land point lying more than 5 m below it is removed; points without a relief value are kept.

After that the class codes are converted to small integers (0 – open sea, 1 – coastal sea, 2 – coast, 3 – inland) and every point is assigned to one of the ten sea regions of the dataset. The points are stored as parallel NumPy arrays of type `float64` (coordinates) and `int8` (classes): 147 763 points occupy about 2.4 MB without the region names, and selecting the points of a rectangle is a single vectorised comparison.

The result of loading the dataset of the project is given in table 3.1. The whole file of 116 MB is read and processed in 1.8 s.

Table 3.1 – Result of loading the dataset

Step	Points
Features read from `merged.json`	152 838
Dropped: invalid coordinates	0
Dropped: unknown class	0
Dropped: class contradicts the relief value	5 075
Kept	147 763
including `OPEN_SEA` / `COASTAL_SEA`	54 701 / 36 258
including `NEAR_COAST` / `COASTLINE`	29 271 / 27 533

By regions the kept points are distributed as follows: North Europe – 29 699; South China Sea – 19 449; Japan and North-West Pacific – 18 434; Black Sea and Eastern Mediterranean – 15 865; Southern Africa – 15 774; Persian Gulf and Arabian Sea – 13 359; Red Sea and Gulf of Aden – 11 170; Central America – 9 722; Strait of Malacca and Andaman Sea – 8 624; Bay of Bengal – 4 695. The remaining 972 points are inland samples outside the ten regions.

3.2 Implementation of the graph construction module

The function `build_graph(points, bbox, config)` builds the graph of an area in four vectorised steps. First, the water points of the area become nodes, and the land points of the area enlarged by 0.5° are collected as obstacles, so that land just outside the rectangle is respected.

Second, candidate edges are found with a k-d tree. Searching neighbours directly in degrees would distort distances away from the equator, therefore all points are converted to Cartesian coordinates on the sphere of radius R:

(3.1)

In these coordinates the distance between two points is the chord, which is a monotonic function of the great-circle distance d:

(3.2)

where d_{ch} – chord length, km.

Hence the k nearest neighbours by chord are exactly the k nearest neighbours by great-circle distance, and the limit l_{max} is converted into a chord by formula (3.2). The `cKDTree` of SciPy [27] returns, for every node, its k = 10 nearest water neighbours within 40 km; pairs found from both ends are merged.

Third, the attributes of all candidate edges are computed at once: the length by formula (1.1), the coastal share from the classes of the end nodes, and the distance to land. For the latter, a second k-d tree on the land points returns the land points within «half the length of the edge + 30 km» of its midpoint, and the exact point-to-segment distance is computed for them in Cartesian coordinates; the smallest value is kept.

Fourth, in the visibility mode every candidate edge is tested against the land. Points are placed along the segment every 1 km, and every such sample is given the class of the nearest point of the dataset, water or land. In other words, the labelled points divide the plane into

Voronoi cells, and an edge is rejected if any of its samples falls into the cell of a land point. The core of the test is the following fragment of `segment_visible`:

```
tree = cKDTree(np.vstack((water_xyz, land_xyz)))
n_samples = np.maximum(np.ceil(length / step_km) - 1, 1)
...
pts = a[idx_seg] + (b[idx_seg] - a[idx_seg]) * t[s:e, None]
_, nearest = tree.query(pts)
on_land = nearest >= n_water        # nearest labelled point is land
visible[idx_seg[on_land]] = False
```

The test needs no polygons of the coastline: it uses only the labelled points, so it works in any part of the dataset. Its resolution is limited by the density of the points; a land strip narrower than the spacing of the samples could be missed, which is why the step is set to 1 km by default and can be reduced in the window.

Finally, the edges are added to a NetworkX graph, and its connected components are found and numbered by size. The departure and destination are attached to the nearest nodes; if these lie in different components (for example one of them in a closed bay of the graph), both ends are attached to the nearest nodes of the largest component.

Figure 3.1 shows the two graphs of the same area between the Cape Peninsula and Cape Agulhas. The difference concerns a small number of edges that cut across the capes; in the open sea the two graphs are identical.

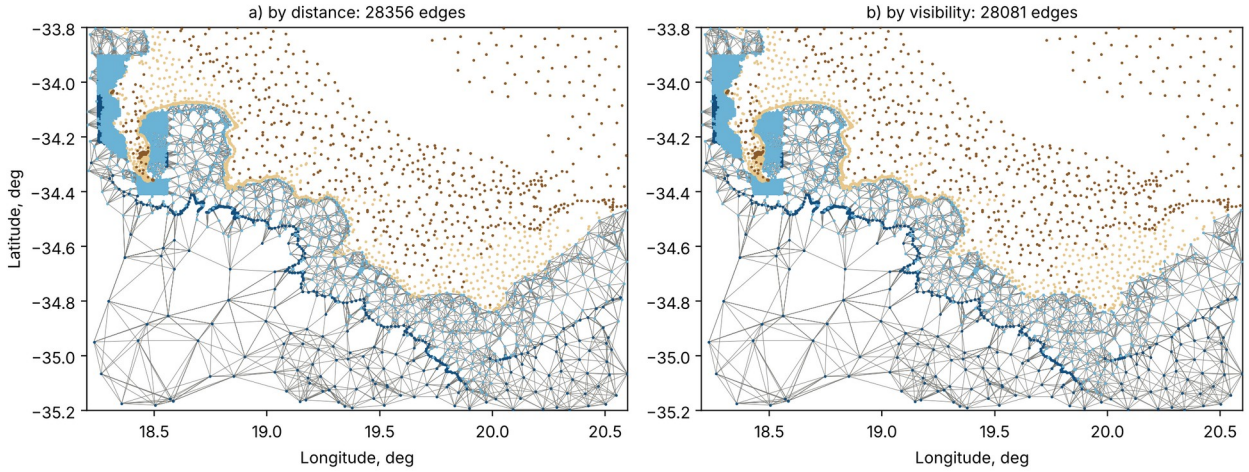


Figure 3.1 – Graph built by distance (a) and by visibility (b) between the Cape Peninsula and Cape Agulhas

The statistics of the graphs of the seven test voyages are given in table 3.2. The area of each graph is the rectangle around the departure and destination with a margin of 2°.

Table 3.2 – Graphs of the test voyages (mode «by visibility», k = 10, l_{max} = 40 km)

Voyage	Nodes	Candidate edges	Removed by visibility	Compo-nents	Mean edge, km	Build time, s	Optimal route of the distance graph across land, edges / km
Odesa – Batumi	6 962	40 693	3 006	54	7.2	1.03	0 / 0.0
Varna – Sinop	5 006	29 372	1 798	13	7.3	0.72	2 / 17.0
Burgas – Trabzon	5 855	34 248	2 698	48	7.7	0.72	0 / 0.0
Sanya – Muara	8 999	50 975	1	2	16.0	0.50	0 / 0.0
Cape Town – Gqeberha	8 372	48 760	715	3	4.5	5.40	1 / 5.5
Dubai – Muscat	3 213	18 164	597	10	9.0	0.25	0 / 0.0
Colon – Limon	3 615	20 825	210	8	6.4	0.55	0 / 0.0

The visibility test removes from 0.002 % to 7.9 % of the candidate edges. The last column shows why it is needed: on the graph built by distance the optimal route of the voyage Varna – Sinop contains 2 legs with a total length of 17.0 km that cross the Sinop peninsula, and the optimal route of the voyage Cape Town – Gqeberha contains one leg of 5.5 km across land (figure 3.2). On the graph by distance such a route looks 4.7 % cheaper than the real optimum, but it cannot be sailed. For this reason the visibility mode is the default.

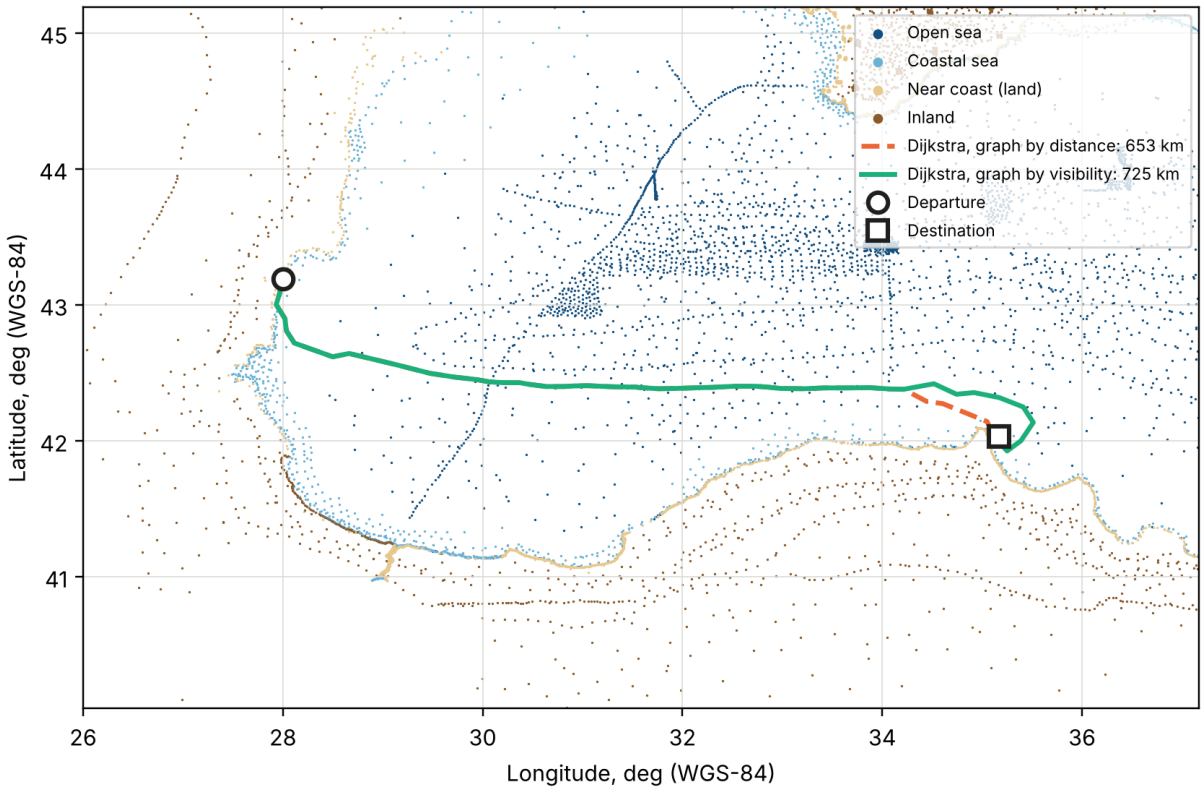


Figure 3.2 – Optimal routes of the voyage Varna – Sinop on the graph built by distance and on the graph built by visibility

The visibility graphs contain several components: small groups of water points in harbours, lagoons and rivers that are separated from the sea by land points. In all test voyages the departure and destination fall into the largest component, which contains from 71 % to 100 % of the nodes.

3.3 Implementation of the genetic algorithm with constraints

The genetic algorithm is implemented in the function `genetic_path(graph, start, goal, config, progress, should_stop)` of the module `genetic.py`. Before the evolution the adjacency lists, the edge costs and the node coordinates are copied from the NetworkX graph into ordinary Python dictionaries (class `_RouteSpace`), because attribute access through NetworkX views is slower and the fitness function is evaluated tens of thousands of times per run. The same class keeps a k-d tree of the nodes for drawing the random intermediate nodes.

The constraints of the problem are handled in three ways:

– the route cannot cross land, because only the edges of the visibility graph can appear in a chromosome;

– the route cannot visit a node twice and is always connected, because every operator either keeps the existing edges or inserts a walk along edges of the graph, and loops are cut out after the crossover;

– coastal water and the vicinity of land are allowed but penalised by the terms $\alpha \cdot L_c$ and $\beta \cdot L_d$ of the fitness function (formula (2.3)).

The random walk of formula (2.5) is the main building block. It is used for the initial population and for the re-routing mutation; in the second case it starts and ends at two nodes of the route, may not enter the other nodes of the route and is limited to $8 \cdot w + 40$ steps, where w is the length of the replaced part. The choice of the next node is implemented as follows:

```
hu = h(u)
weights = []
for v in options:                          # unvisited neighbours of u
    edge = self.cost[u][v] or 1e-9
    gain = (hu - h(v)) / edge              # progress per unit of cost
    weights.append(math.exp(kappa * gain))
v = rng.choices(options, weights=weights, k=1)[0]
```

The crossover looks for the common inner nodes of the parents with a set intersection, which takes time proportional to the length of the routes:

```
common = list(set(a[1:-1]) & set(b[1:-1]))
if not common:
    return list(a), list(b)
node = rng.choice(common)
i, j = a.index(node), b.index(node)
return _remove_loops(a[:i] + b[j:]), _remove_loops(b[:j] + a[i:])
```

The evolution loop sorts the population by cost, copies the e best routes, and fills the rest of the new population with the children of tournament winners. After every generation the callback `progress(generation, best, median)` is called; in the application it emits the signal that updates the status bar and the progress indicator. The callback `should_stop()` is checked at the same point, which makes the button «Stop» effective within one generation.

All random decisions use one object `random.Random(config.seed)`. The test of reproducibility runs the algorithm twice with the same seed and compares the routes; they coincide exactly. The number of fitness evaluations in a run equals $N \cdot (G + 1)$, where G is the number of generations actually run: 30 100 evaluations for N = 100 and G = 300.

Dijkstra's and A* algorithms (module `search.py`) were written by the author for the regular grid of the diploma project and are ported to the point graph without changes of their logic: the 8-neighbourhood of a grid cell is replaced by the adjacency list of a node, and the table of step lengths by the edge attribute `weight`. The priority queue is the binary heap of the module `heapq` [31]. Since `heapq` has no decrease-key operation, an improved distance is pushed as a new entry and outdated entries are skipped when they are taken from the heap (lazy deletion). The search stops when the destination is taken from the heap, and the path is restored from the predecessor links.

3.4 Testing, comparison with Dijkstra and A*, analysis of the results

Testing was carried out in two parts: automated tests of the modules and experiments on the full dataset. All runs were made under Ubuntu 24.04 with Python 3.13 on a virtual machine with an Intel Xeon processor at 2.1 GHz; every algorithm runs in one thread.

Automated tests. The course project has 15 tests written with pytest (table 3.3). Most of them use a small synthetic dataset: a water area of 1°×1° divided by a wall of land points with a single strait in the north, so the correct answer is known in advance. Together with the 19 tests of the shared modules (geodesy and file reading) all 34 tests pass.

Table 3.3 – Automated tests of the course project

Test	What is checked
`test_xyz_is_on_the_sphere`	Cartesian points lie on the sphere of radius R
`test_land_points_are_not_nodes`	only water classes become nodes
`test_visibility_removes_edges_across_the_wall`	no visibility edge crosses the wall
`test_edge_attributes`	length > 0, coastal share ∈ {0; 0.5; 1}, c(e) ≥ l(e)
`test_cost_model`	danger factor and edge cost on hand-computed values
`test_dijkstra_and_astar_match_networkx`	both costs equal the NetworkX reference; A* expands no more nodes; the path goes through the strait
`test_unreachable_goal_is_reported`	all three algorithms report «no path»
`test_remove_loops`	cutting of loops after the crossover
`test_genetic_route_is_valid_and_not_better_than_optimum`	simple connected route, $F^* \le F \le 1.1 \cdot F^*$, best cost never increases
`test_genetic_is_reproducible`	the same seed gives the same route
`test_route_breakdown_adds_up`	$F = L + \alpha \cdot L_c + \beta \cdot L_d$
`test_load_points_cleans_conflicts`	counters of the loading steps on a small file
`test_session_and_export`	session, comparison table and all export files
`test_route_area_and_presets`	area around a route, coordinates of the presets
`test_main_window_starts_offscreen`	the main window is created, GA defaults are shown

Test voyages. Seven voyages in five sea areas of the dataset were chosen as presets of the application and as test cases (table 3.4). They include an open-sea passage (Sanya – Muara), passages across an enclosed sea with capes near the destination (Black Sea), a long coastal passage around Cape Agulhas (Cape Town – Gqeberha) and passages through straits (Hormuz).

Table 3.4 – Test voyages

Voyage	Departure φ, λ, deg	Destination φ, λ, deg	Distance to the nearest node, km	Length of the optimal route, km	Nodes of the optimal route
Odesa – Batumi	46.45, 30.80	41.65, 41.60	0.3 / 1.7	1 094.6	75
Varna – Sinop	43.19, 28.00	42.03, 35.17	1.3 / 3.1	724.8	48
Burgas – Trabzon	42.48, 27.55	41.02, 39.75	0.6 / 1.1	1 093.2	62
Sanya – Muara	18.20, 109.50	5.00, 114.95	5.7 / 18.6	1 623.4	89
Cape Town – Gqeberha	−33.90, 18.40	−33.95, 25.65	0.7 / 1.0	873.0	198
Dubai – Muscat	25.30, 55.25	23.65, 58.60	0.3 / 2.2	586.2	59
Colon – Limon	9.40, −79.90	10.00, −83.02	1.3 / 3.1	388.4	36

On six voyages the end points are attached to nodes not farther than 3.1 km. On the voyage Sanya – Muara the departure is 5.7 km and the destination 18.6 km from the nearest water point of the dataset, so this route ends offshore of the port. The application writes a warning to the log when an end point is farther than 25 km from the graph.

Comparison of the algorithms. On every voyage Dijkstra's and A* algorithms were run on the visibility graph with the default weights $\alpha = 0.5$, $\beta = 2$, $r = 10$ km, and the genetic

algorithm was run ten times with the seeds 0–9 and the default parameters of table 2.2. The running time of the exact searches is the median of five runs. The results are given in table 3.5.

Table 3.5 – Results of Dijkstra's, A* and the genetic algorithm

Voyage	Optimal cost F*	Nodes expanded, Dijkstra / A*	Time, ms, Dijkstra / A*	GA gap δ, %: mean (min…max)	GA runs with δ ≤ 1 %	GA time, s
Odesa – Batumi	1 280.3	5 940 / 3 465	31 / 27	0.28 (0.13…0.52)	10 of 10	4.2
Varna – Sinop	836.4	4 478 / 2 483	18 / 15	2.07 (0.07… 10.07)	7 of 10	2.2
Burgas – Trabzon	1 220.2	5 155 / 4 113	24 / 31	0.16 (0.00…0.46)	10 of 10	3.1
Sanya – Muara	1 636.0	8 688 / 1 926	48 / 13	0.28 (0.08…0.99)	10 of 10	3.1
Cape Town – Gqeberha	956.3	8 007 / 7 279	38 / 49	2.85 (2.08…4.58)	0 of 10	5.7
Dubai – Muscat	694.6	2 804 / 1 289	12 / 8	0.28 (0.12…0.35)	10 of 10	3.3
Colon – Limon	626.0	2 319 / 1 885	10 / 11	0.37 (0.16…0.70)	10 of 10	2.5

The results allow the following conclusions to be drawn.

1) Dijkstra's and A* algorithms give the same optimal cost on all seven voyages, as follows from the consistency of the heuristic. A* expands on average 1.97 times fewer nodes (from 1.10 to 4.51 times; figure 3.3). The gain is largest on the open-sea voyage Sanya – Muara, where the cost of a route is close to its length and the heuristic is accurate, and smallest on the coastal voyage Cape Town – Gqeberha, where the route goes around the cape and the straight-line estimate is far from the real cost. In time the gain is smaller: both searches take 8–49 ms, and on three voyages A* is even slightly slower, because the computation of the heuristic costs more than the saved expansions.

2) The genetic algorithm found a valid route in all 70 runs. In 57 runs of 70 its cost is within 1 % of the optimum; the mean gap over all runs is 0.90 %, the median gap is 0.26 %. On five voyages every run is within 1 %.

3) The hardest voyage for the genetic algorithm is Cape Town – Gqeberha: the optimal route has 198 nodes and follows the coast, and the gap is 2.08–4.58 %. On Varna – Sinop seven runs come within 0.08 % of the optimum, while three runs (3.2, 6.9, 10.1 %) settle on a corridor with more coastal water: in two of them the route leaves Varna to the north-east along the coast, in the third it approaches Sinop along the western shore of the peninsula, which is about 19 km shorter than the optimal route but runs close to land. Once the population has lost the better corridor, the mutations do not bring it back before the stagnation limit is reached.

4) The genetic algorithm needs 2.2–5.7 s per run, about a hundred times more than the exact searches, because it evaluates 24 340–30 100 complete routes instead of expanding a few thousand nodes.

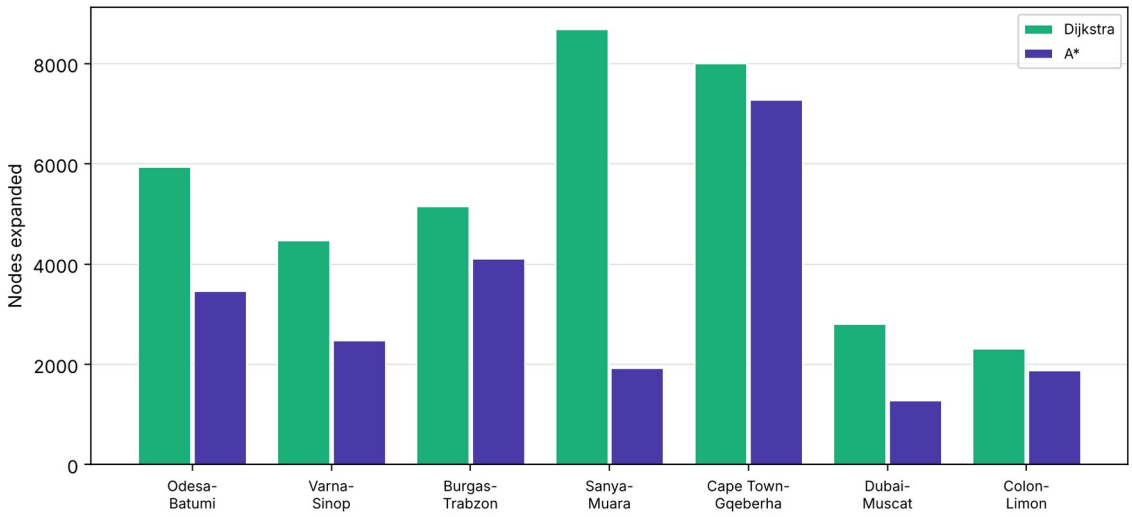


Figure 3.3 – Number of nodes expanded by Dijkstra's and A* algorithms

The routes of the voyage Cape Town – Gqeberha are shown in figure 3.4. The route of the genetic algorithm (seed 0) coincides with the optimal one on most of the passage and differs from it mainly near Cape Town. It is shorter than the optimal route (861.5 km against 873.0 km) but has a higher cost (976.2 against 956.3), because it passes closer to land: its danger-weighted length is 36.5 km against 20.2 km. Under the chosen criterion the shorter route is therefore the worse one.

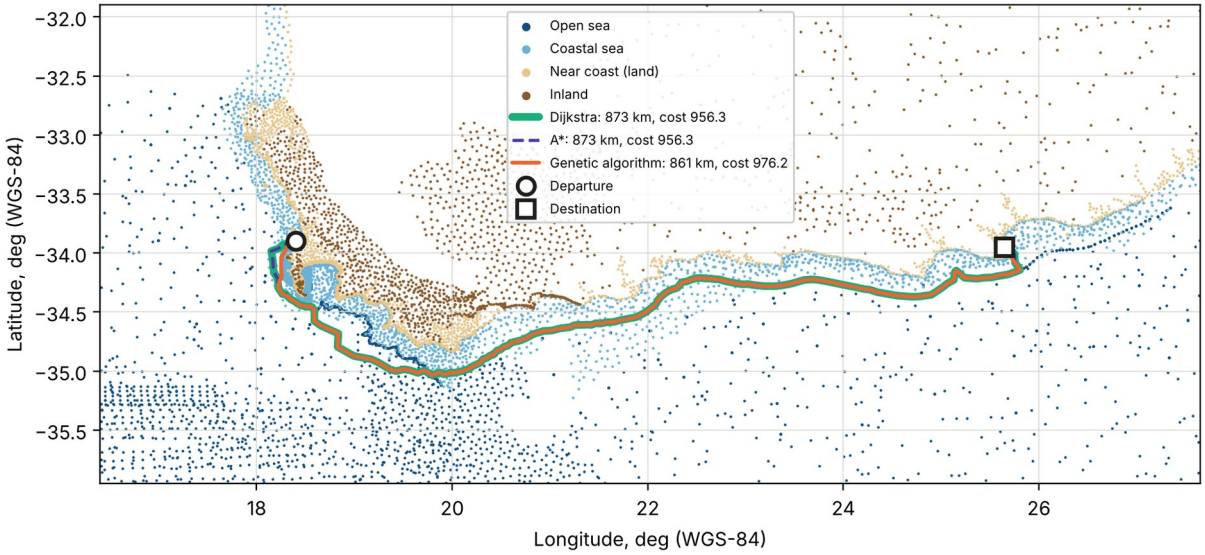


Figure 3.4 – Routes of the voyage Cape Town – Gqeberha

The convergence curves of two reference runs are shown in figure 3.5. In the run for Odesa – Batumi the best cost falls from 1 374.0 in the initial population to 1 284.2; the algorithm stops after 231 generations because the best cost has not improved for 80 generations. For Cape Town – Gqeberha the initial routes are much worse (best cost 1 863.5), the improvement continues almost to the limit of 300 generations, and the final cost 976.2 remains 2.08 % above the optimum. After the first few dozen generations the median cost of

the population follows the best cost closely, i.e. most routes of the population become similar to the best one.

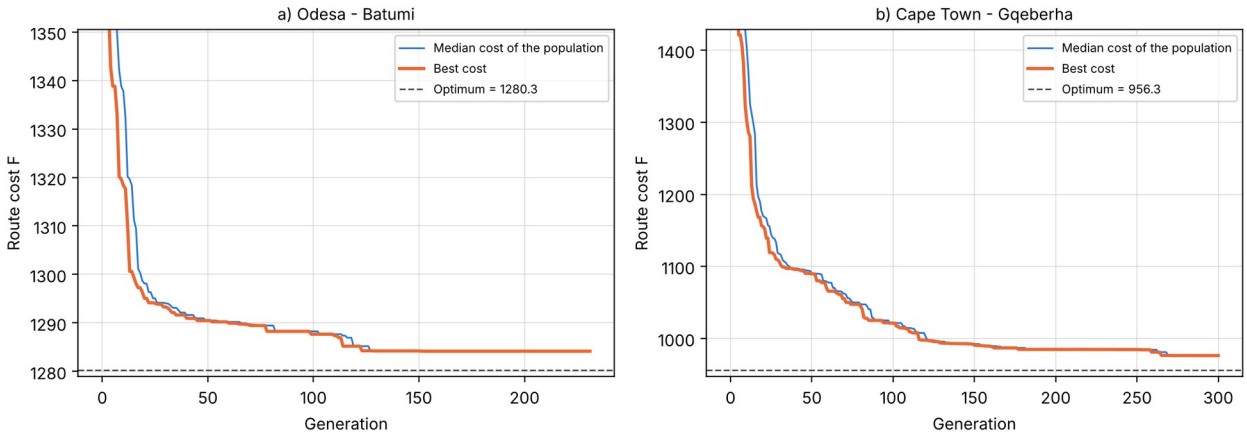


Figure 3.5 – Convergence of the genetic algorithm: a) Odesa – Batumi; b) Cape Town – Gqeberha

Influence of the penalties. To check that the penalties work as intended, the optimal route of the voyage Cape Town – Gqeberha was computed for five combinations of α and β (table 3.6).

Table 3.6 – Optimal route of the voyage Cape Town – Gqeberha for different penalty weights (r = 10 km)

α	β	Length L, km	Coastal L_c, km	Danger-weighted L_d, km	Closest approach to land, km
0	0	792.1	584.4	205.0	0.07
0.5	0	846.1	90.4	50.3	0.94
0	2	853.4	332.5	18.9	0.94
0.5	2	873.0	85.8	20.2	0.94
2	5	918.0	46.0	18.3	0.94

Without penalties the shortest route is 792 km long, but 584 km of it lie in coastal water and it passes 68 m from a land point. Each penalty moves the route offshore; with the default weights the length grows by 10.2 %, the length in coastal water falls from 584 to 86 km and the closest approach to land increases to 0.94 km. A further increase of the weights reduces the coastal length but not the closest approach: it is reached on the first leg of the route, at the departure in Table Bay, which no route can avoid.

Sensitivity to the parameters. The parameters of the genetic algorithm were varied one at a time on the two hardest voyages, five runs per value (seeds 0–4). The results are given in table 3.7 and figure 3.6.

Table 3.7 – Sensitivity of the genetic algorithm: mean gap δ, % (maximum in brackets) and mean time, s

Parameter	Value	Cape Town – Gqeberha: δ, %	Time, s	Varna – Sinop: δ, %	Time, s
Population size N	20	11.79 (14.95)	1.1	5.98 (10.78)	0.5
	50	8.06 (11.44)	2.9	4.69 (9.91)	1.4
	100	2.63 (3.04)	5.4	0.07 (0.07)	2.7
	150	2.69 (4.28)	8.7	0.72 (3.38)	4.0
Mutation probability p_m	0.1	6.41 (9.56)	3.5	4.36 (9.79)	1.2
	0.3	2.63 (3.04)	5.5	0.07 (0.07)	2.7
	0.5	3.28 (4.47)	8.7	1.99 (6.64)	3.4
	0.7	2.71 (4.16)	10.9	0.05 (0.07)	4.7
Crossover probability p_c	0.5	3.00 (4.63)	5.5	3.20 (9.71)	2.5
	0.8	2.63 (3.04)	5.7	0.07 (0.07)	2.6
	0.95	2.58 (3.86)	6.0	0.73 (3.44)	2.7
Operators	full operator set	2.63 (3.04)	5.7	0.07 (0.07)	2.8
	without intermediate nodes	12.50 (15.50)	5.2	1.84 (3.23)	2.1
	without shortcut mutation	2.63 (3.06)	7.7	2.07 (6.64)	3.7
	without crossover	2.34 (2.62)	4.9	3.36 (9.85)	2.7

The following regularities can be seen:

– a population of 20 or 50 routes is too small: the mean gap is 11.8 and 8.1 % on the coastal voyage. From N = 100 the result hardly improves, while the time grows in proportion to N, so N = 100 is kept as the default;

– a mutation probability of 0.1 gives too little variation (gap 6.4 and 4.4 %); the values 0.3–0.7 are clearly better, without a consistent trend among them, while the time grows with p_m, because most mutations call a random walk;

– on the coastal voyage the crossover probability in the range 0.5–0.95 changes the mean gap by less than 0.5 %, whereas on Varna – Sinop p_c = 0.5 gives 3.2 % against 0.07 % at p_c = 0.8;

– the intermediate nodes of the random walk are the most important part of the operator set: without them the mean gap on the coastal voyage grows from 2.6 to 12.5 %, because without them the walks rarely leave the straight line between their end nodes and the population does not cover the corridor along the coast;

– the crossover and the shortcut mutation help on the voyage Varna – Sinop (without them the mean gap grows to 3.4 and 2.1 %), but on the coastal voyage they make no significant difference; with five runs per variant, differences of a few tenths of a per cent between the variants are within the scatter of the results.

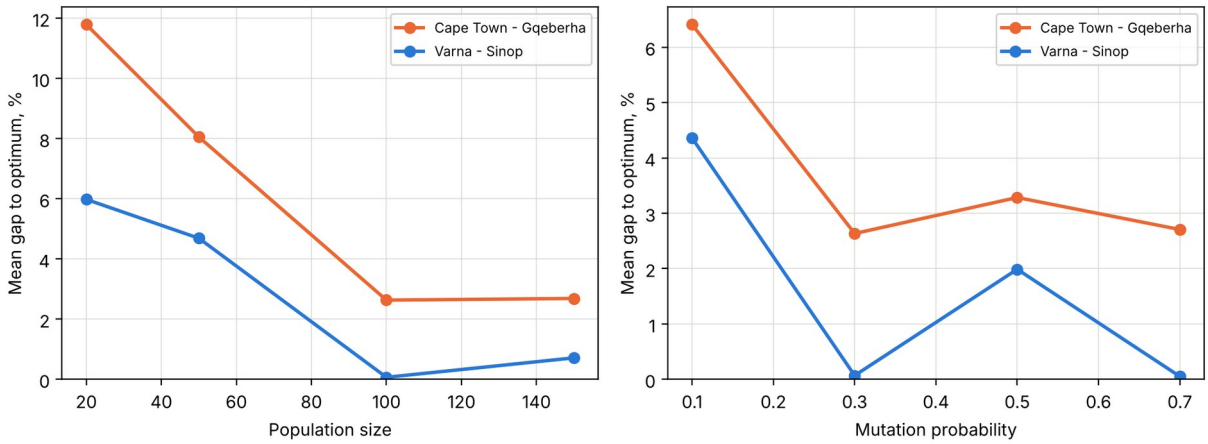


Figure 3.6 – Mean gap of the genetic algorithm as a function of the population size and of the mutation probability

Check of the user interface. The window was tested with the scenario of Appendix A: opening the dataset, choosing a preset, building the graph with and without the display of the edges, running each algorithm separately and all three together, stopping the genetic algorithm, changing the penalties and the parameters and exporting the results. The export folder receives eight files: the map and the convergence plot (PNG), the routes (GeoJSON and CSV), the comparison table (CSV), the parameter log (JSON and TXT) and the interactive map (HTML). The screenshots in Appendix A were taken from this run.

Cross-platform operation. The application uses only the standard library of Python and packages that are distributed for Windows and Linux (table 1.3); file paths are formed with `pathlib`, and no system-specific calls are made. The dependencies of the course project are listed in the file `requirements-course.txt`, and the installation for both systems is described in Appendix A.

Analysis of the results. On the point graph the genetic algorithm reliably finds navigable routes and in most runs comes within 1 % of the optimum, but it is slower than the exact searches by two orders of magnitude and, on long coastal passages, stops a few per cent above the optimum. This agrees with the general property of evolutionary search on problems with an additive criterion: it does not use the structure that makes Dijkstra's and A* algorithms exact. The value of the genetic algorithm in the application is its flexibility. Its fitness function is an ordinary function of the whole route and can be extended without changes to the search, while the exact searches computed on the same graph provide a reference that shows how much the evolutionary solution loses.

CONCLUSION

In the course project the desktop application «Sea Course GA» was developed. It finds the course of a sea vessel over the open-sea points of the author's labelled dataset with a genetic algorithm and compares the result with Dijkstra's and A* algorithms on the same graph.

The following results were obtained.

1) The subject area and the methods of route search were analysed. The course is represented as a sequence of water points of the dataset, the dangers as the points of the coastal water and land classes. The choice of the genetic algorithm with the path encoding of variable length and of the tools (Python, NetworkX, SciPy, PyQt5, Matplotlib) was justified.

2) A loading module was implemented that reads GeoJSON, CSV and JSON files and removes points whose class contradicts the relief value. Of the 152 838 points of the dataset 5 075 (3.32 %) were removed by this rule.

3) A graph module was implemented that joins every water point with its nearest neighbours and removes the edges that cross land, using only the labelled points. Without this test the optimal routes of two of the seven test voyages cross land.

4) The problem was formalised: the cost of a route is its length plus penalties for the length sailed in coastal water and close to land; the chromosome is the route itself; the operators are a goal-directed random walk with intermediate nodes, tournament selection, crossover at a common node, re-routing and shortcut mutations and elitism.

5) Testing on seven voyages showed that Dijkstra's and A* algorithms give the same optimum, with A* expanding on average 1.97 times fewer nodes. The genetic algorithm found a navigable route in all 70 runs, in 57 of them within 1 % of the optimum, with a mean gap of 0.90 %. The hardest case is the long coastal voyage Cape Town – Gqeberha with a gap of 2.1–4.6 %.

6) The study of the parameters showed that a population of at least 100 routes, a mutation probability of 0.3 and the use of random intermediate nodes in the walks are necessary for a good result.

All requirements of the assignment are met: the application loads the student's dataset, builds the graph by distance or by visibility, lets the user set the parameters of the genetic algorithm and the penalties, shows the map with the three routes and the convergence curve, and saves the map, the routes and the parameter log to files.

The application can be developed further in two directions. The fitness function can be extended with criteria that the exact searches cannot handle, such as the number of course changes or the time of the passage at a given speed. The quality of the genetic algorithm on long coastal passages can be improved by seeding the initial population with routes found on a coarser graph or by a local search applied to the best route.

LIST OF ABBREVIATIONS

A* – A-star, heuristic best-first search algorithm
CSV – Comma-Separated Values
GA – genetic algorithm
GeoJSON – format of geographic data based on JSON
GUI – graphical user interface
HTML – HyperText Markup Language
IMO – International Maritime Organization
JSON – JavaScript Object Notation
k-d tree – k-dimensional tree
PNG – Portable Network Graphics
WGS-84 – World Geodetic System 1984

REFERENCES

1. International Maritime Organization. Guidelines for voyage planning : Resolution A.893(21) adopted on 25 November 1999 [Electronic resource]. – London : IMO, 1999. – URL: https://wwwcdn.imo.org/localresources/en/KnowledgeCentre/IndexofIMOResolutions/AssemblyDocuments/A.893(21).pdf (date of access: 06.10.2026).

2. Bowditch, N. The American Practical Navigator : Pub. No. 9. Vol. I / N. Bowditch ; National Geospatial-Intelligence Agency. – 2019 ed. – Springfield, VA : NGA, 2019.

3. Dijkstra, E. W. A note on two problems in connexion with graphs / E. W. Dijkstra // Numerische Mathematik. – 1959. – Vol. 1, No. 1. – P. 269–271. – DOI 10.1007/BF01386390.

4. Hart, P. E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. – 1968. – Vol. 4, No. 2. – P. 100–107. – DOI 10.1109/TSSC.1968.300136.

5. Holland, J. H. Adaptation in Natural and Artificial Systems : An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence / J. H. Holland. – Cambridge, MA : MIT Press, 1992. – 211 p. – ISBN 0-262-58111-6.

6. Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning / D. E. Goldberg. – Reading, MA : Addison-Wesley, 1989. – 412 p. – ISBN 0-201-15767-5.

7. Department of Defense World Geodetic System 1984 : Its Definition and Relationships with Local Geodetic Systems : NIMA TR8350.2 / National Imagery and Mapping Agency. – 3rd ed., Amendment 1. – Bethesda, MD : NIMA, 2000.

8. Sinnott, R. W. Virtues of the Haversine / R. W. Sinnott // Sky and Telescope. – 1984. – Vol. 68, No. 2. – P. 159.

9. Tam, C. K. Review of Collision Avoidance and Path Planning Methods for Ships in Close Range Encounters / C. K. Tam, R. Bucknall, A. Greig // Journal of Navigation. – 2009. – Vol. 62, No. 3. – P. 455–476. – DOI 10.1017/S0373463308005134.

10. Walther, L. Modeling and Optimization Algorithms in Ship Weather Routing / L. Walther, A. Rizvanolli, M. Wendebourg, C. Jahn // International Journal of e-Navigation and Maritime Economy. – 2016. – Vol. 4. – P. 31–45. – DOI 10.1016/j.enavi.2016.06.004.

11. Zis, T. P. V. Ship weather routing: A taxonomy and survey / T. P. V. Zis, H. N. Psaraftis, L. Ding // Ocean Engineering. – 2020. – Vol. 213. – Art. 107697. – DOI 10.1016/j.oceaneng.2020.107697.

12. Cormen, T. H. Introduction to Algorithms / T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. – 3rd ed. – Cambridge, MA : MIT Press, 2009. – 1292 p. – ISBN 978-0-262-03384-8.

13. Russell, S. Artificial Intelligence : A Modern Approach / S. Russell, P. Norvig. – 4th ed. – Hoboken, NJ : Pearson, 2020. – 1136 p. – ISBN 978-0-13-461099-3.

14. Liu, C. An Improved A-Star Algorithm Considering Water Current, Traffic Separation and Berthing for Vessel Path Planning / C. Liu, Q. Mao, X. Chu, S. Xie // Applied Sciences. – 2019. – Vol. 9, No. 6. – Art. 1057. – DOI 10.3390/app9061057.

15. Lozano-Pérez, T. An algorithm for planning collision-free paths among polyhedral obstacles / T. Lozano-Pérez, M. A. Wesley // Communications of the ACM. – 1979. – Vol. 22, No. 10. – P. 560–570. – DOI 10.1145/359156.359164.

16. Zaccone, R. Ship voyage optimization for safe and energy-efficient navigation: A dynamic programming approach / R. Zaccone, E. Ottaviani, M. Figari, M. Altosole // Ocean Engineering. – 2018. – Vol. 153. – P. 215–224. – DOI 10.1016/j.oceaneng.2018.01.100.

17. Mitchell, M. An Introduction to Genetic Algorithms / M. Mitchell. – Cambridge, MA : MIT Press, 1996. – 205 p. – ISBN 0-262-13316-4.

18. Eiben, A. E. Introduction to Evolutionary Computing / A. E. Eiben, J. E. Smith. – 2nd ed. – Berlin ; Heidelberg : Springer, 2015. – DOI 10.1007/978-3-662-44874-8.

19. Гладков, Л. А. Генетические алгоритмы : учебник / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2010. – 368 с. – ISBN 978-5-9221-0510-1.

20. Miller, B. L. Genetic Algorithms, Tournament Selection, and the Effects of Noise / B. L. Miller, D. E. Goldberg // Complex Systems. – 1995. – Vol. 9, No. 3. – P. 193–212.

21. Coello Coello, C. A. Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art / C. A. Coello Coello // Computer Methods in Applied Mechanics and Engineering. – 2002. – Vol. 191, No. 11–12. – P. 1245–1287. – DOI 10.1016/S0045-7825(01)00323-1.

22. Ahn, C. W. A genetic algorithm for shortest path routing problem and the sizing of populations / C. W. Ahn, R. S. Ramakrishna // IEEE Transactions on Evolutionary Computation. – 2002. – Vol. 6, No. 6. – P. 566–579. – DOI 10.1109/TEVC.2002.804323.

23. Śmierzchalski, R. Modeling of ship trajectory in collision situations by an evolutionary algorithm / R. Śmierzchalski, Z. Michalewicz // IEEE Transactions on Evolutionary Computation. – 2000. – Vol. 4, No. 3. – P. 227–241. – DOI 10.1109/4235.873234.

24. Szłapczyńska, J. Multi-objective Weather Routing with Customised Criteria and Constraints / J. Szłapczyńska // Journal of Navigation. – 2015. – Vol. 68, No. 2. – P. 338–354. – DOI 10.1017/S0373463314000691.

25. Hagberg, A. A. Exploring network structure, dynamics, and function using NetworkX / A. A. Hagberg, D. A. Schult, P. J. Swart // Proceedings of the 7th Python in Science Conference (SciPy2008) / ed. by G. Varoquaux, T. Vaught, J. Millman. – Pasadena, CA, 2008. – P. 11–15.

26. Harris, C. R. Array programming with NumPy / C. R. Harris, K. J. Millman, S. J. van der Walt [et al.] // Nature. – 2020. – Vol. 585, No. 7825. – P. 357–362. – DOI 10.1038/s41586-020-2649-2.

27. Virtanen, P. SciPy 1.0: fundamental algorithms for scientific computing in Python / P. Virtanen, R. Gommers, T. E. Oliphant [et al.] // Nature Methods. – 2020. – Vol. 17, No. 3. – P. 261–272. – DOI 10.1038/s41592-019-0686-2.

28. PyQt5 5.15.11 : Python bindings for the Qt cross platform application toolkit [Electronic resource] / Riverbank Computing Limited. – URL: https://pypi.org/project/PyQt5/ (date of access: 06.10.2026).

29. Hunter, J. D. Matplotlib: A 2D Graphics Environment / J. D. Hunter // Computing in Science & Engineering. – 2007. – Vol. 9, No. 3. – P. 90–95. – DOI 10.1109/MCSE.2007.55.

30. Bentley, J. L. Multidimensional binary search trees used for associative searching / J. L. Bentley // Communications of the ACM. – 1975. – Vol. 18, No. 9. – P. 509–517. – DOI 10.1145/361002.361007.

31. heapq – Heap queue algorithm // The Python Standard Library : Python 3 documentation [Electronic resource] / Python Software Foundation. – URL: https://docs.python.org/3/library/heapq.html (date of access: 06.10.2026).

APPENDIX A
User guide

A.1 Purpose and requirements. The application «Sea Course GA» finds the course of a sea vessel between two positions over the water points of a labelled dataset with a genetic algorithm and compares it with the optimal routes of Dijkstra's and A* algorithms. It requires Windows 10/11 or Linux (Ubuntu 20.04+, Fedora 38+) and Python 3.10 or newer; on Ubuntu 20.04, where the system Python is 3.8, Python 3.10 has to be installed additionally. A computer with at least 4 GB of RAM is recommended: reading the dataset of the project needs about 2 GB of memory at its peak.

A.2 Installation. The folder of the project is copied to the computer, and the dependencies are installed into a virtual environment. Under Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-course.txt
```

Under Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-course.txt
```

If Qt reports on Ubuntu that the platform plugin «xcb» cannot be loaded, the package `libxcb-xinerama0` has to be installed. The dataset is placed in the folder `data/raw` under the name `merged.json`.

A.3 Start. The application is started from the folder of the project:

```
python scripts/run_desktop.py
python scripts/run_desktop.py --dataset path/to/points.geojson
```

Without the option `--dataset` the file `data/raw/merged.json` is opened automatically if it exists. Another file is opened with the menu «File → Open dataset...» (Ctrl+O) or the button «Open...». The file must contain the latitude, the longitude and the attribute `class_code` with one of the values `OPEN_SEA`, `COASTAL_SEA`, `NEAR_COAST`, `COASTLINE`.

A.4 Work with the application. A typical session consists of the following steps.

1) Check in the group «1. Dataset» that the file is loaded and the number of points of every class is shown. The check box «Remove class/depth conflicts (5 m)» should normally stay on.

2) In the group «2. Route» choose a preset voyage or type the coordinates of the departure and the destination in decimal degrees. The positions can also be picked on the map: press «Pick departure» or «Pick destination» and click on the map.

3) In the group «3. Graph» keep the area «Around the route» and the edge mode «By visibility». Press «Build graph». The check box «Show edges» draws the edges on the map (figure A.1).

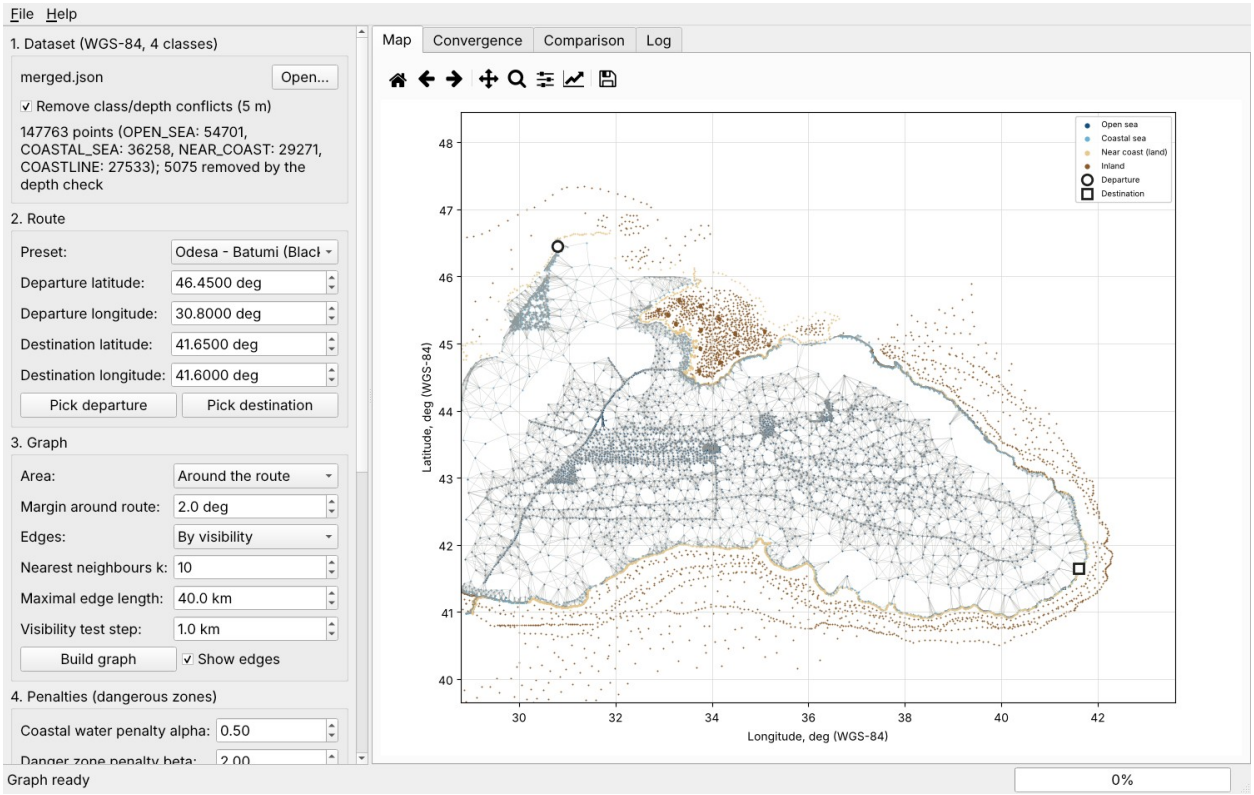


Figure A.1 – Main window after building the graph

4) In the group «4. Penalties» set the coastal water penalty α, the danger zone penalty β and the danger zone radius r. A change of these values recomputes the edge costs and clears the previous results.

5) In the group «5. Genetic algorithm» set the parameters of the algorithm (figure A.2). The same random seed gives the same route.

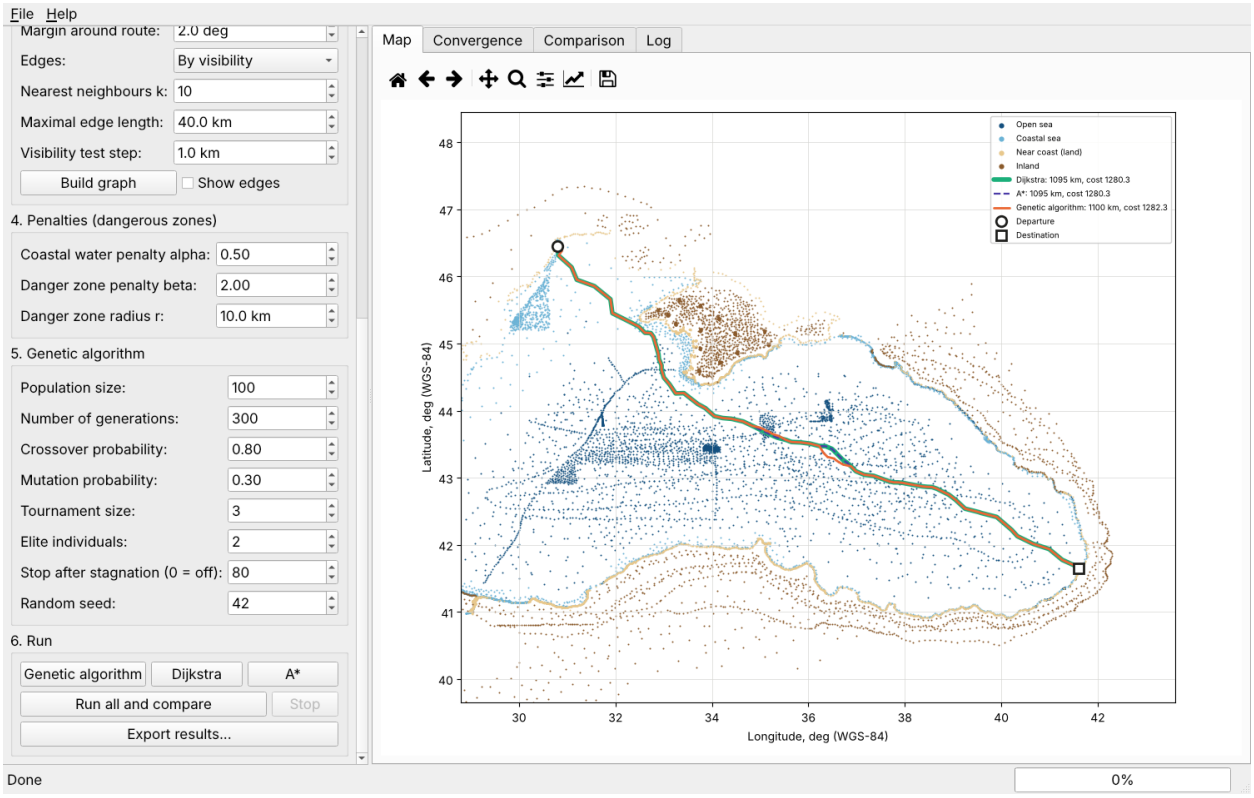


Figure A.2 – Groups «Penalties», «Genetic algorithm» and «Run» of the control panel

6) Press «Run all and compare» or one of the buttons «Genetic algorithm», «Dijkstra», «A*». The status bar shows the current generation and the best cost; «Stop» ends the genetic algorithm after the current generation. If the graph is out of date, it is rebuilt first.

7) Read the results on the tabs «Map» (figure A.3), «Convergence» (figure A.4) and «Comparison» (figure A.5). The column «Gap, %» shows the excess of the cost over the optimum of Dijkstra/A*.

8) Press «Export results...» (Ctrl+E) and choose a folder; the files are listed in table A.1.

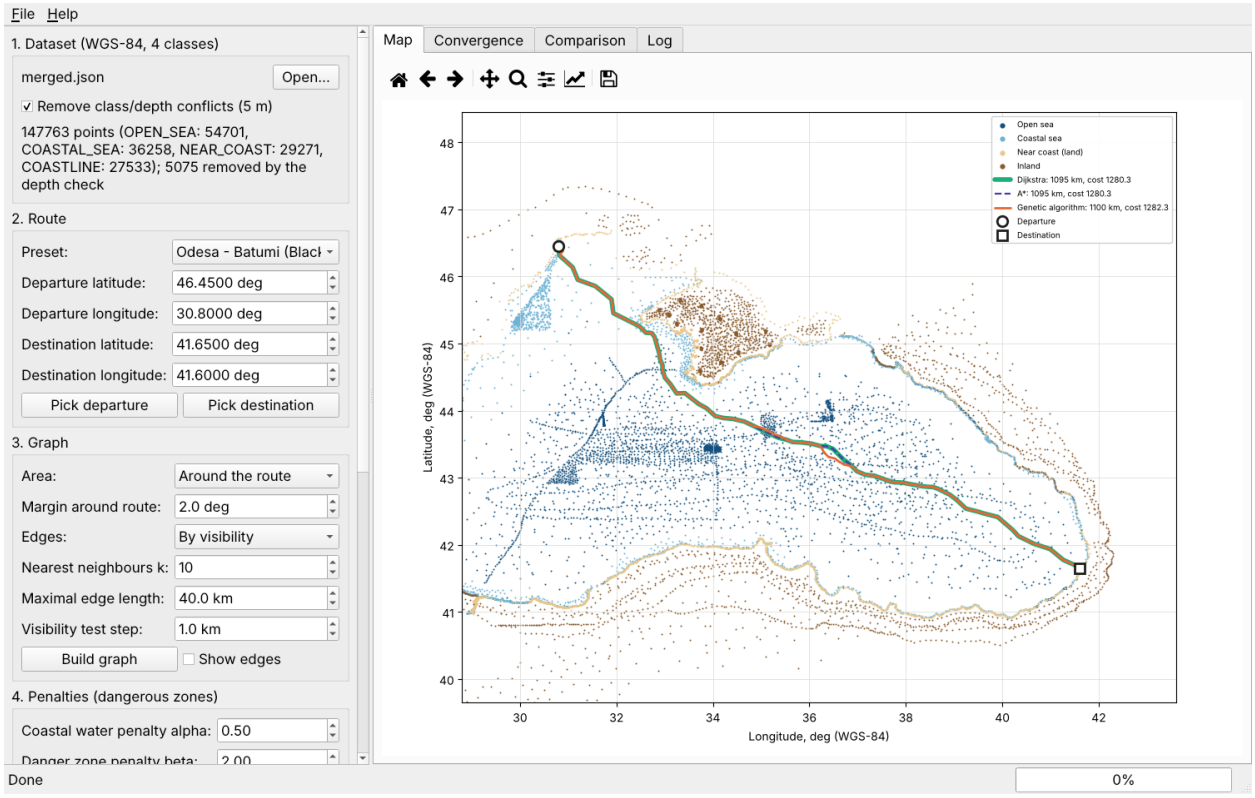


Figure A.3 – Routes of the voyage Odesa – Batumi on the tab «Map»

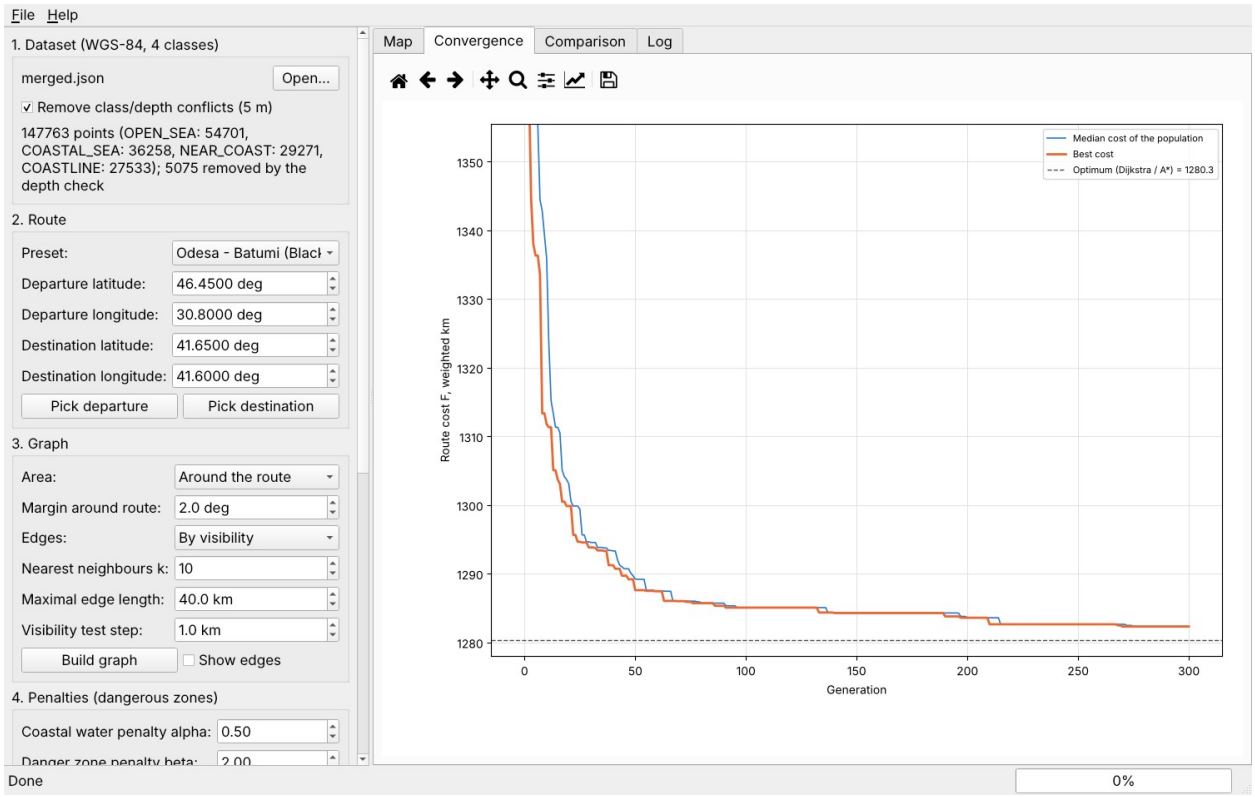


Figure A.4 – Tab «Convergence»

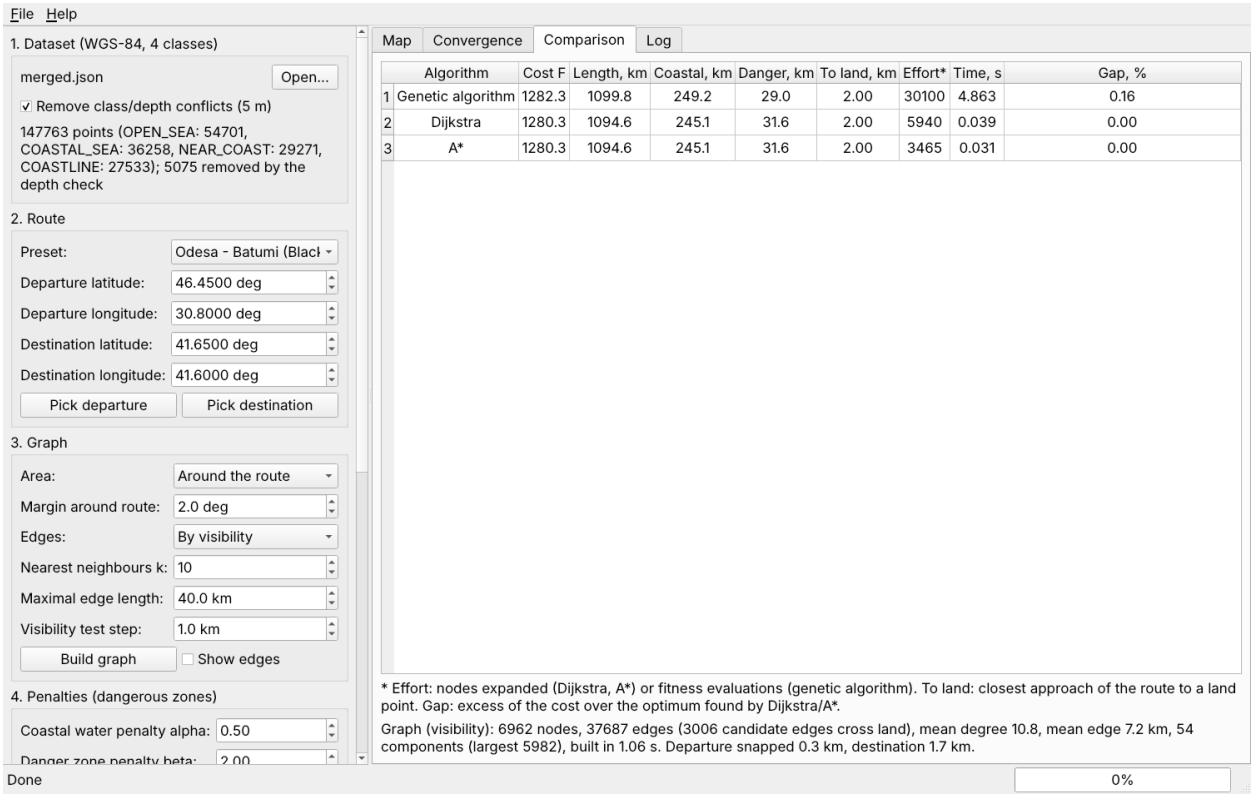


Figure A.5 – Tab «Comparison»

Table A.1 – Files written by the export (prefix «course_<date>_<time>»)

File	Content
`<prefix>_map.png`	map with the points and the routes
`<prefix>_convergence.png`	convergence plot of the genetic algorithm
`<prefix>_routes.geojson`	routes as GeoJSON lines, departure and destination as points
`<prefix>_routes.csv`	waypoints of every route: algorithm, number, latitude, longitude
`<prefix>_comparison.csv`	comparison table of the algorithms
`<prefix>_log.json`	all parameters and results in machine-readable form
`<prefix>_log.txt`	the same in readable form
`<prefix>_map.html`	interactive map (only if folium is installed)

A.5 Messages. The messages of the application and the actions of the user are listed in table A.2. The full text of every error, including the technical details, is written to the tab «Log».

Table A.2 – Messages of the application

Message	Cause and action
«Open the dataset first.»	a graph or a search was requested before loading a file; open a dataset
«The file has no 'class_code' attribute – a labelled dataset is required»	the file has no classes; use a labelled dataset
«No labelled points remain after validation»	no valid point in the file; check the coordinates and the class codes
«The selected area contains fewer than two water points»	the area has no water; move the end points or increase the margin
«No path: start and goal lie in different components»	the end points cannot be joined on this graph; increase the margin or the maximal edge length
«Warning: an end point is far from the nearest graph node»	the dataset has few water points near the end point; the route starts or ends at the nearest node
«Run at least one algorithm first.»	export without results; run an algorithm

Appendix B

Educational Institution «Brest State Technical University»
Department of Intelligent Information Technologies

SEA COURSE GA

«Development of an application for finding the optimal course of a sea vessel over open-sea points based on a genetic algorithm»

PROGRAM TEXT

КР.ИИ2025.250978-01 12 00

Sheets 21

Developed by: student of group ИИ-2025 _____ Li Haoran

Checked by: _____ A. A. Kozinsky

Brest 2026

ABSTRACT

The document contains the source text of the key modules of the program «Sea Course GA» – the application for finding the optimal course of a sea vessel over open-sea points based on a genetic algorithm. The program loads a labelled dataset of WGS-84 points, builds a navigation graph, searches the course with a genetic algorithm and compares it with Dijkstra's and A* algorithms.

The program is written in Python (version 3.10 or newer) and consists of the package `maritime_route.pointgraph` (search), the package `maritime_route.desktop` (user interface) and helper modules. The listing contains the modules of the package `maritime_route.pointgraph` that implement loading of the data, the construction of the graph, the cost model, the exact searches and the genetic algorithm, 1371 lines in total. The complete source code is supplied in electronic form.

CONTENTS

1 Module dataset.py (214 lines)..... 3
2 Module graph.py (407 lines)..... 6
3 Module cost.py (100 lines)..... 11
4 Module search.py (213 lines)..... 13
5 Module genetic.py (437 lines)..... 16

1 Module dataset.py

```
"""Labelled point set used by the course-project route search.

The course project works directly on the labelled corpus: every point keeps
its WGS-84 coordinates and one of the four zone classes, and no neural network
is involved. This module turns a GeoJSON/CSV/JSON file into a compact
:class:`PointSet` of NumPy arrays and selects the part of it that lies inside
a rectangular area of interest.

Reused parts of the diploma code base
-------------------------------------
* :func:`maritime_route.data.ais_loader.load_any` and
  :func:`~maritime_route.data.ais_loader.validate` read the file and drop rows
  with missing or out-of-range coordinates;
* :func:`maritime_route.data.cleaning.conflict_mask` removes points whose class
  contradicts the measured height or depth (tolerance 5 m);
* :func:`maritime_route.data.cleaning.assign_regions` maps points to the ten
  sea regions of the corpus.

Class codes
-----------
The integer class index is the position in
:data:`~maritime_route.config.CLASS_NAMES`:

=====  ==============  ============================================
index  code            role in the route search
=====  ==============  ============================================
0      OPEN_SEA        graph node, no penalty
1      COASTAL_SEA     graph node, penalised (shallow water)
2      NEAR_COAST      obstacle (land within 5 km of the shore)
3      COASTLINE       obstacle (land remote from the sea)
=====  ==============  ============================================
"""
from __future__ import annotations

import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import numpy as np
import pandas as pd

from ..config import CLASS_NAMES, CLASS_TO_INDEX, LABEL_CHECK_TOLERANCE_M, REGIONS
from ..data.ais_loader import DataLoadError, load_any, validate
from ..data.cleaning import assign_regions, conflict_mask

#: Integer indices of the navigable classes (graph nodes).
WATER_INDICES: Tuple[int, ...] = (CLASS_TO_INDEX["OPEN_SEA"], CLASS_TO_INDEX["COASTAL_SEA"])
#: Integer indices of the land classes (obstacles).
LAND_INDICES: Tuple[int, ...] = (CLASS_TO_INDEX["NEAR_COAST"], CLASS_TO_INDEX["COASTLINE"])

#: Bounding box ``(lat_min, lat_max, lon_min, lon_max)`` in degrees.
BBox = Tuple[float, float, float, float]


@dataclass
class LoadSummary:
    """What happened while a point file was read.

    Attributes
    ----------
    source:
        File name.
    n_read:
        Features found in the file.
    n_invalid:
        Rows dropped because of missing or out-of-range coordinates.
    n_unknown_class:
        Rows without one of the four known class codes.
    n_conflicts:
        Rows removed by the label/measurement consistency check.
    n_points:
        Points kept in the :class:`PointSet`.
    class_counts:
        Number of kept points per class code.
    seconds:
        Wall-clock loading time.
    """

    source: str
    n_read: int
    n_invalid: int
```

```
    n_unknown_class: int
    n_conflicts: int
    n_points: int
    class_counts: Dict[str, int] = field(default_factory=dict)
    seconds: float = 0.0

    def as_text(self) -> str:
        """Multi-line description for the log window of the GUI."""
        lines = [
            f"File: {self.source}",
            f"Features read: {self.n_read}",
            f"Dropped (invalid coordinates): {self.n_invalid}",
            f"Dropped (unknown class): {self.n_unknown_class}",
            f"Dropped (class contradicts depth/height): {self.n_conflicts}",
            f"Points kept: {self.n_points}",
        ]
        lines += [f"  {name}: {count}" for name, count in self.class_counts.items()]
        lines.append(f"Loading time: {self.seconds:.1f} s")
        return "\n".join(lines)


@dataclass
class PointSet:
    """Labelled points as parallel NumPy arrays.

    Attributes
    ----------
    lat, lon:
        WGS-84 coordinates, decimal degrees, float64.
    cls:
        Class index 0..3 (see the module docstring), int8.
    region:
        Name of the sea region of every point (object array).
    """

    lat: np.ndarray
    lon: np.ndarray
    cls: np.ndarray
    region: np.ndarray

    def __len__(self) -> int:
        return int(self.lat.shape[0])

    def select(self, mask: np.ndarray) -> "PointSet":
        """Subset given by a boolean mask (arrays are copied)."""
        return PointSet(self.lat[mask].copy(), self.lon[mask].copy(),
                        self.cls[mask].copy(), self.region[mask].copy())

    def in_bbox(self, bbox: BBox, margin_deg: float = 0.0) -> np.ndarray:
        """Boolean mask of the points inside ``bbox`` enlarged by ``margin_deg``."""
        lat_min, lat_max, lon_min, lon_max = bbox
        return ((self.lat >= lat_min - margin_deg) & (self.lat <= lat_max + margin_deg)
                & (self.lon >= lon_min - margin_deg) & (self.lon <= lon_max + margin_deg))

    def class_counts(self) -> Dict[str, int]:
        """Number of points per class code, in the order of ``CLASS_NAMES``."""
        counts = np.bincount(self.cls.astype(np.int64), minlength=len(CLASS_NAMES))
        return {name: int(counts[i]) for i, name in enumerate(CLASS_NAMES)}


def load_points(path: Path | str, clean: bool = True,
                tolerance_m: float = LABEL_CHECK_TOLERANCE_M) -> Tuple[PointSet, LoadSummary]:
    """Read a labelled point file into a :class:`PointSet`.

    Parameters
    ----------
    path:
        GeoJSON FeatureCollection (``.geojson``/``.json`` with ``features``),
        CSV or JSON array. Every record needs ``latitude``, ``longitude`` and
        ``class_code``; a GeoJSON point geometry overrides the coordinate
        properties.
    clean:
        Remove points whose class contradicts their measured height/depth.
    tolerance_m:
        Tolerance of that check, metres.

    Returns
    -------
    (PointSet, LoadSummary)

    Raises
    ------
    DataLoadError
        If the file has no usable labelled points.
    """
```

```
    t0 = time.perf_counter()
    path = Path(path)
    frame = load_any(path)
    n_read = len(frame)
    frame, _report = validate(frame, source=path.name)
    n_invalid = n_read - len(frame)
    if "class_code" not in frame.columns:
        raise DataLoadError("The file has no 'class_code' attribute - a labelled dataset is required")
    known = frame["class_code"].isin(CLASS_NAMES)
    n_unknown = int((~known).sum())
    frame = frame.loc[known].reset_index(drop=True)
    n_conflicts = 0
    if clean and "noaa_depth" in frame.columns:
        bad = conflict_mask(frame, tolerance_m)
        n_conflicts = int(bad.sum())
        frame = frame.loc[~bad].reset_index(drop=True)
    if frame.empty:
        raise DataLoadError("No labelled points remain after validation")
    points = frame_to_points(frame)
    summary = LoadSummary(
        source=path.name, n_read=n_read, n_invalid=n_invalid,
        n_unknown_class=n_unknown, n_conflicts=n_conflicts, n_points=len(points),
        class_counts=points.class_counts(), seconds=time.perf_counter() - t0,
    )
    return points, summary


def frame_to_points(frame: pd.DataFrame) -> PointSet:
    """Convert a validated, labelled DataFrame into a :class:`PointSet`."""
    lat = frame["latitude"].to_numpy(dtype=np.float64)
    lon = frame["longitude"].to_numpy(dtype=np.float64)
    cls = frame["class_code"].map(CLASS_TO_INDEX).to_numpy(dtype=np.int8)
    region = assign_regions(lat, lon)
    return PointSet(lat, lon, cls, region)


def region_names() -> List[str]:
    """Names of the predefined sea regions of the corpus."""
    return [name for name, _ in REGIONS]


def region_bbox(name: str) -> Optional[BBox]:
    """Bounding box of a predefined region, or ``None`` for an unknown name."""
    for region, box in REGIONS:
        if region == name:
            return box
    return None
```

2 Module graph.py

```
"""Navigation graph built directly on the labelled water points.

Nodes
-----
Every OPEN_SEA or COASTAL_SEA point inside the area of interest becomes a
node. Land points (NEAR_COAST, COASTLINE) never become nodes; they are the
obstacles of the problem.

Edges
-----
Candidate edges join each node with its ``k`` nearest water neighbours that
lie closer than ``max_edge_km``. Two construction modes are offered:

* ``"distance"`` - every candidate edge is kept. The graph is purely
  geometric; an edge may pass over a narrow cape or island, which is then only
  discouraged by the danger penalty of the cost model.
* ``"visibility"`` - a candidate edge is kept only if the segment between its
  end points "sees" no land. The segment is sampled every ``sample_step_km``
  kilometres and every sample is labelled with the class of its nearest
  dataset point (1-nearest-neighbour rule, i.e. the labelled points split the
  plane into Voronoi cells). If any sample falls into the cell of a land
  point, the edge crosses land and is discarded.

For every edge three attributes are stored, so that the costs can be
recomputed for new penalty weights without rebuilding the graph:

* ``length`` - great-circle length, km;
* ``coastal`` - share of the edge in coastal water, 0, 0.5 or 1 (mean of the
  two end-point flags);
* ``land_km`` - distance from the segment to the nearest land point, km
  (``inf`` when no land point is within the search radius).

Geometry
--------
Points are converted to 3-D Cartesian coordinates on a sphere of radius
6371.0088 km. A k-d tree on these coordinates returns neighbours in exact
great-circle order, because the chord length ``2R sin(d / 2R)`` grows
monotonically with the arc length ``d``. The point-to-segment distance is
computed in the same 3-D space; for segments of a few tens of kilometres the
difference between the chord and the arc is below 0.01 %.
"""
from __future__ import annotations

import math
import time
from dataclasses import asdict, dataclass
from typing import Callable, Dict, List, Optional, Tuple

import networkx as nx
import numpy as np
from scipy.spatial import cKDTree

from ..config import EARTH_RADIUS_KM
from ..routing.geodesy import haversine_km
from .dataset import LAND_INDICES, WATER_INDICES, BBox, PointSet

#: ``progress(message, fraction)`` callback used by the GUI.
ProgressFn = Optional[Callable[[str, float], None]]


@dataclass(frozen=True)
class GraphConfig:
    """Parameters of graph construction.

    Attributes
    ----------
    mode:
        ``"distance"`` or ``"visibility"`` (see the module docstring).
    k_neighbours:
        Number of nearest water neighbours tested for every node.
    max_edge_km:
        Longest admissible edge, km.
    sample_step_km:
        Visibility mode: spacing of the test samples along an edge, km.
    land_search_km:
        Edge-to-land distances are computed up to this value; farther land is
        recorded as ``inf``. It must not be smaller than the danger radius of
        the cost model (the GUI limits the radius accordingly).
    margin_deg:
        Margin added around the area of interest when land points are
        collected, so that obstacles just outside the area are respected.
    """
```

```
    mode: str = "visibility"
    k_neighbours: int = 10
    max_edge_km: float = 40.0
    sample_step_km: float = 1.0
    land_search_km: float = 30.0
    margin_deg: float = 0.5

    def to_dict(self) -> Dict:
        return asdict(self)


@dataclass
class GraphStats:
    """Summary of a built graph, shown in the GUI and written to the log."""

    n_water_points: int
    n_land_points: int
    n_candidate_edges: int
    n_edges: int
    n_removed_by_visibility: int
    n_components: int
    largest_component: int
    mean_degree: float
    mean_edge_km: float
    seconds: float

    def to_dict(self) -> Dict:
        return asdict(self)


@dataclass
class SeaGraph:
    """The navigation graph and the arrays it was built from.

    Attributes
    ----------
    graph:
        Undirected :class:`networkx.Graph`; node ids are ``0..n-1``; node
        attributes ``lat``, ``lon``, ``cls``; edge attributes ``length``,
        ``coastal``, ``land_km`` and, after
        :func:`~maritime_route.pointgraph.cost.apply_costs`, ``weight``.
    lat, lon, cls:
        Node coordinates (degrees) and classes, indexed by node id.
    land_lat, land_lon:
        Land points of the area, used for drawing and by the tests.
    bbox:
        Area of interest.
    config:
        Construction parameters.
    stats:
        Summary numbers.
    component:
        Connected-component label of every node (0 = largest component).
    """

    graph: nx.Graph
    lat: np.ndarray
    lon: np.ndarray
    cls: np.ndarray
    land_lat: np.ndarray
    land_lon: np.ndarray
    bbox: BBox
    config: GraphConfig
    stats: GraphStats
    component: np.ndarray
    _tree: Optional[cKDTree] = None

    @property
    def n_nodes(self) -> int:
        return int(self.lat.shape[0])

    def coords(self, path: List[int]) -> List[Tuple[float, float]]:
        """``(lat, lon)`` of the nodes of a path."""
        return [(float(self.lat[i]), float(self.lon[i])) for i in path]

    def nearest_node(self, lat: float, lon: float,
                     largest_only: bool = False) -> Tuple[int, float]:
        """Node closest to a position and its great-circle distance (km).

        Parameters
        ----------
        lat, lon:
            Position, degrees.
        largest_only:
```

```
            Restrict the search to the largest connected component.
        """
        if self._tree is None:
            self._tree = cKDTree(to_xyz(self.lat, self.lon))
        query = to_xyz(np.array([lat]), np.array([lon]))[0]
        if not largest_only:
            _, idx = self._tree.query(query)
        else:
            k = min(self.n_nodes, 64)
            while True:
                _, cand = self._tree.query(query, k=k)
                cand = np.atleast_1d(cand)
                hit = cand[self.component[cand] == 0]
                if hit.size or k >= self.n_nodes:
                    idx = int(hit[0]) if hit.size else int(cand[0])
                    break
                k = min(self.n_nodes, k * 4)
        idx = int(idx)
        return idx, float(haversine_km(lat, lon, self.lat[idx], self.lon[idx]))


def to_xyz(lat: np.ndarray, lon: np.ndarray) -> np.ndarray:
    """Cartesian coordinates (km) of points on the sphere of radius R."""
    phi = np.radians(np.asarray(lat, dtype=np.float64))
    lam = np.radians(np.asarray(lon, dtype=np.float64))
    cos_phi = np.cos(phi)
    return EARTH_RADIUS_KM * np.column_stack((cos_phi * np.cos(lam), cos_phi * np.sin(lam), np.sin(phi)))


def chord_from_arc(arc_km: float) -> float:
    """Chord length that corresponds to a great-circle distance (km)."""
    return 2.0 * EARTH_RADIUS_KM * math.sin(min(arc_km, math.pi * EARTH_RADIUS_KM) / (2.0 * EARTH_RADIUS_KM))


def segment_land_distance(a: np.ndarray, b: np.ndarray, land_xyz: np.ndarray,
                          land_tree: Optional[cKDTree], search_km: float,
                          chunk: int = 20000) -> np.ndarray:
    """Distance from every segment ``a[i] - b[i]`` to the nearest land point.

    Only land points within ``half length + search_km`` of the segment
    midpoint are examined; when there is none the distance is ``inf``.

    Parameters
    ----------
    a, b:
        ``(m, 3)`` Cartesian end points of the segments, km.
    land_xyz:
        ``(p, 3)`` Cartesian land points, km.
    land_tree:
        k-d tree of ``land_xyz`` (``None`` when there are no land points).
    search_km:
        Extra search radius around each segment.
    chunk:
        Number of segments processed per vectorised batch.

    Returns
    -------
    numpy.ndarray
        ``(m,)`` distances in km.
    """
    m = a.shape[0]
    out = np.full(m, np.inf)
    if land_tree is None or m == 0:
        return out
    for s in range(0, m, chunk):
        e = min(m, s + chunk)
        aa, bb = a[s:e], b[s:e]
        mid = 0.5 * (aa + bb)
        radius = 0.5 * np.linalg.norm(bb - aa, axis=1) + search_km
        hits = land_tree.query_ball_point(mid, radius)
        counts = np.fromiter((len(h) for h in hits), dtype=np.int64, count=e - s)
        if counts.sum() == 0:
            continue
        seg = np.repeat(np.arange(e - s), counts)
        pts = np.concatenate([np.asarray(h, dtype=np.int64) for h in hits if len(h)])
        p = land_xyz[pts]
        ab = (bb - aa)[seg]
        ap = p - aa[seg]
        denom = np.einsum("ij,ij->i", ab, ab)
        t = np.where(denom > 0, np.einsum("ij,ij->i", ap, ab) / np.where(denom > 0, denom, 1.0), 0.0)
        t = np.clip(t, 0.0, 1.0)
        closest = aa[seg] + ab * t[:, None]
        dist = np.linalg.norm(p - closest, axis=1)
        best = np.full(e - s, np.inf)
        np.minimum.at(best, seg, dist)
```

```
        out[s:e] = best
    return out


def segment_visible(a: np.ndarray, b: np.ndarray, length: np.ndarray,
                    water_xyz: np.ndarray, land_xyz: np.ndarray, step_km: float,
                    chunk: int = 200000) -> np.ndarray:
    """Line-of-sight test of segments over the labelled point map.

    Interior samples are placed every ``step_km`` along each segment; a sample
    is "land" when its nearest labelled point (water or land) is a land point.
    A segment is visible when none of its samples is land.

    Parameters
    ----------
    a, b:
        ``(m, 3)`` Cartesian end points, km.
    length:
        ``(m,)`` segment lengths, km.
    water_xyz, land_xyz:
        Cartesian coordinates of the water and land points, km.
    step_km:
        Sampling step, km.
    chunk:
        Number of samples classified per batch.

    Returns
    -------
    numpy.ndarray of bool
        ``(m,)``, True for a segment that does not cross land.
    """
    m = a.shape[0]
    visible = np.ones(m, dtype=bool)
    if m == 0 or land_xyz.shape[0] == 0:
        return visible
    tree = cKDTree(np.vstack((water_xyz, land_xyz)))
    n_water = water_xyz.shape[0]
    n_samples = np.maximum(np.ceil(length / max(step_km, 1e-3)).astype(np.int64) - 1, 1)
    seg = np.repeat(np.arange(m), n_samples)
    # Position of every sample inside its segment: k / (n + 1), k = 1..n.
    first = np.cumsum(n_samples) - n_samples
    k = np.arange(seg.size) - np.repeat(first, n_samples) + 1
    t = k / (np.repeat(n_samples, n_samples) + 1.0)
    for s in range(0, seg.size, chunk):
        e = min(seg.size, s + chunk)
        idx_seg = seg[s:e]
        pts = a[idx_seg] + (b[idx_seg] - a[idx_seg]) * t[s:e, None]
        _, nearest = tree.query(pts)
        on_land = nearest >= n_water
        visible[idx_seg[on_land]] = False
    return visible


def build_graph(points: PointSet, bbox: BBox, config: GraphConfig = GraphConfig(),
                progress: ProgressFn = None) -> SeaGraph:
    """Build the navigation graph of an area.

    Parameters
    ----------
    points:
        Whole labelled dataset.
    bbox:
        Area of interest ``(lat_min, lat_max, lon_min, lon_max)``.
    config:
        Construction parameters.
    progress:
        Optional ``progress(message, fraction)`` callback.

    Returns
    -------
    SeaGraph

    Raises
    ------
    ValueError
        If the area contains fewer than two water points or the mode is unknown.
    """
    if config.mode not in ("distance", "visibility"):
        raise ValueError(f"Unknown graph mode: {config.mode}")
    t0 = time.perf_counter()

    def report(message: str, fraction: float) -> None:
        if progress is not None:
            progress(message, fraction)
```

```
inside = points.in_bbox(bbox)
water_mask = inside & np.isin(points.cls, WATER_INDICES)
land_mask = points.in_bbox(bbox, config.margin_deg) & np.isin(points.cls, LAND_INDICES)
lat, lon, cls = points.lat[water_mask], points.lon[water_mask], points.cls[water_mask]
land_lat, land_lon = points.lat[land_mask], points.lon[land_mask]
n = lat.shape[0]
if n < 2:
    raise ValueError("The selected area contains fewer than two water points")
report(f"{n} water points, {land_lat.shape[0]} land points", 0.05)

# 1. Candidate edges: k nearest neighbours within max_edge_km.
xyz = to_xyz(lat, lon)
tree = cKDTree(xyz)
k = min(config.k_neighbours + 1, n)
dist, idx = tree.query(xyz, k=k, distance_upper_bound=chord_from_arc(config.max_edge_km))
rows = np.repeat(np.arange(n), k - 1)
cols = idx[:, 1:].ravel()
ok = cols < n                       # missing neighbours are reported as index n
rows, cols = rows[ok], cols[ok]
lo, hi = np.minimum(rows, cols), np.maximum(rows, cols)
pairs = np.unique(np.column_stack((lo, hi)), axis=0)
pairs = pairs[pairs[:, 0] != pairs[:, 1]]
n_candidates = int(pairs.shape[0])
report(f"{n_candidates} candidate edges", 0.25)

# 2. Edge attributes.
u, v = pairs[:, 0], pairs[:, 1]
length = haversine_km(lat[u], lon[u], lat[v], lon[v])
coastal_flag = (cls == WATER_INDICES[1]).astype(np.float64)
coastal = 0.5 * (coastal_flag[u] + coastal_flag[v])
land_xyz = to_xyz(land_lat, land_lon) if land_lat.size else np.zeros((0, 3))
land_tree = cKDTree(land_xyz) if land_lat.size else None
search = config.land_search_km
land_km = segment_land_distance(xyz[u], xyz[v], land_xyz, land_tree, search)
report("Distances to land computed", 0.75)

# 3. Visibility filter.
keep = np.ones(n_candidates, dtype=bool)
if config.mode == "visibility" and land_lat.size:
    keep = segment_visible(xyz[u], xyz[v], length, xyz, land_xyz, config.sample_step_km)
removed = int((~keep).sum())

graph = nx.Graph()
graph.add_nodes_from(
    (i, {"lat": float(lat[i]), "lon": float(lon[i]), "cls": int(cls[i])}) for i in range(n)
)
graph.add_edges_from(
    (int(a), int(b), {"length": float(d), "coastal": float(c), "land_km": float(l)})
    for a, b, d, c, l in zip(u[keep], v[keep], length[keep], coastal[keep], land_km[keep])
)

# 4. Connected components, label 0 = largest.
components = sorted(nx.connected_components(graph), key=len, reverse=True)
component = np.empty(n, dtype=np.int64)
for label, nodes in enumerate(components):
    component[list(nodes)] = label
n_edges = graph.number_of_edges()
stats = GraphStats(
    n_water_points=n, n_land_points=int(land_lat.shape[0]),
    n_candidate_edges=n_candidates, n_edges=n_edges,
    n_removed_by_visibility=removed, n_components=len(components),
    largest_component=len(components[0]) if components else 0,
    mean_degree=2.0 * n_edges / n,
    mean_edge_km=float(length[keep].mean()) if n_edges else 0.0,
    seconds=time.perf_counter() - t0,
)
report("Graph ready", 1.0)
return SeaGraph(graph=graph, lat=lat, lon=lon, cls=cls, land_lat=land_lat,
                land_lon=land_lon, bbox=bbox, config=config, stats=stats,
                component=component, _tree=tree)
```

3 Module cost.py

```
"""Cost model shared by the genetic algorithm, Dijkstra and A*.

The cost of an edge ``e = (u, v)`` is its length increased by two penalties::

    c(e) = l(e) * (1 + alpha * s(e) + beta * phi(e))

    phi(e) = max(0, 1 - d_land(e) / r)

where

* ``l(e)`` - great-circle length of the edge, km;
* ``s(e)`` - share of the edge in coastal (shallow) water, 0 / 0.5 / 1;
* ``d_land(e)`` - distance from the edge to the nearest land point, km;
* ``r`` - radius of the danger zone around land, km;
* ``alpha`` - coastal-water penalty, ``beta`` - danger-zone penalty.

``phi`` is 1 for an edge that touches land and falls linearly to 0 at the
edge of the danger zone. The cost of a route is the sum of its edge costs,
which splits into the three terms used as the fitness of the genetic
algorithm::

    F(P) = L(P) + alpha * L_c(P) + beta * L_d(P)

with the route length ``L``, the length sailed in coastal water ``L_c`` and
the danger-weighted length ``L_d``. Because every factor is at least 1, the
cost of an edge is never smaller than its length, which keeps the great-circle
heuristic of A* admissible.
"""
from __future__ import annotations

import math
from dataclasses import asdict, dataclass
from typing import Dict, List

import networkx as nx

from .graph import SeaGraph


@dataclass(frozen=True)
class CostWeights:
    """Penalty weights of the cost model.

    Attributes
    ----------
    coastal_penalty:
        ``alpha`` - extra cost per km sailed in coastal water (0.5 = +50 %).
    danger_penalty:
        ``beta`` - extra cost per km sailed directly next to land.
    danger_radius_km:
        ``r`` - width of the danger zone around land points, km.
    """

    coastal_penalty: float = 0.5
    danger_penalty: float = 2.0
    danger_radius_km: float = 10.0

    def to_dict(self) -> Dict:
        return asdict(self)


def danger_factor(land_km: float, radius_km: float) -> float:
    """``phi`` - 1 at the shore, 0 outside the danger zone."""
    if radius_km <= 0 or not math.isfinite(land_km):
        return 0.0
    return max(0.0, 1.0 - land_km / radius_km)


def edge_cost(length: float, coastal: float, land_km: float, weights: CostWeights) -> float:
    """Cost ``c(e)`` of one edge (weighted km)."""
    phi = danger_factor(land_km, weights.danger_radius_km)
    return length * (1.0 + weights.coastal_penalty * coastal + weights.danger_penalty * phi)


def apply_costs(sea: SeaGraph, weights: CostWeights) -> None:
    """Write the attribute ``weight`` (edge cost) on every edge of the graph."""
    for _, _, data in sea.graph.edges(data=True):
        data["weight"] = edge_cost(data["length"], data["coastal"], data["land_km"], weights)


def route_breakdown(graph: nx.Graph, path: List[int], weights: CostWeights) -> Dict[str, float]:
    """Terms of the fitness function for a node path.
```

```
Returns
-------
dict
    ``length_km`` (L), ``coastal_km`` (L_c), ``danger_km`` (L_d),
    ``cost`` (F) and ``min_land_km`` (closest approach to land).
"""
length = coastal = danger = 0.0
min_land = math.inf
for a, b in zip(path[:-1], path[1:]):
    data = graph.edges[a, b]
    length += data["length"]
    coastal += data["length"] * data["coastal"]
    danger += data["length"] * danger_factor(data["land_km"], weights.danger_radius_km)
    min_land = min(min_land, data["land_km"])
cost = length + weights.coastal_penalty * coastal + weights.danger_penalty * danger
return {"length_km": length, "coastal_km": coastal, "danger_km": danger,
        "cost": cost, "min_land_km": min_land}
```

4 Module search.py

```
"""Dijkstra and A* on the point graph.

Both searches were first written by the author for the regular lattice of the
diploma project (:mod:`maritime_route.routing.dijkstra` and
:mod:`maritime_route.routing.astar`). Here they are ported to an arbitrary
weighted graph: the 8-neighbourhood of a lattice cell is replaced by the
adjacency list of a node, and the pre-computed step-length table by the edge
attribute ``weight``. The structure of the algorithms is unchanged:

* the open list is a binary heap with lazy deletion (``heapq`` has no
  decrease-key, so an improved distance is pushed as a new entry and stale
  entries are skipped when popped);
* a node is final the first time it is popped (all costs are positive and the
  A* heuristic is consistent);
* the search stops as soon as the goal is popped.

A* uses ``h(n) = d_gc(n, goal)``, the great-circle distance to the goal. Every
edge cost is at least the edge length (see :mod:`.cost`), and the sum of the
edge lengths of any path is at least the great-circle distance between its
ends, so ``h`` never over-estimates the remaining cost (admissible) and
satisfies ``h(u) <= c(u, v) + h(v)`` (consistent). A* therefore returns the
same optimal cost as Dijkstra while expanding fewer nodes.
"""
from __future__ import annotations

import heapq
import math
import time
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

import networkx as nx

from ..config import EARTH_RADIUS_KM


@dataclass
class PathResult:
    """Outcome of one route search (any of the three algorithms).

    Attributes
    ----------
    algorithm:
        ``"genetic"``, ``"dijkstra"`` or ``"astar"``.
    found:
        True if a path from the start to the goal was found.
    path:
        Node ids from the start to the goal.
    cost:
        Sum of edge costs (weighted km); ``inf`` if not found.
    expanded:
        Dijkstra/A*: nodes taken from the heap. GA: fitness evaluations.
    generated:
        Dijkstra/A*: heap insertions. GA: generations run.
    runtime_s:
        Wall-clock time, seconds.
    best_history, median_history:
        GA only - best and median population cost after every generation.
    message:
        Human-readable outcome.
    """

    algorithm: str
    found: bool
    path: List[int] = field(default_factory=list)
    cost: float = math.inf
    expanded: int = 0
    generated: int = 0
    runtime_s: float = 0.0
    best_history: List[float] = field(default_factory=list)
    median_history: List[float] = field(default_factory=list)
    message: str = ""


def _reconstruct(came_from: Dict[int, int], goal: int) -> List[int]:
    """Follow predecessor links back from the goal and reverse the list."""
    path = [goal]
    while path[-1] in came_from:
        path.append(came_from[path[-1]])
    path.reverse()
    return path
```

```
def dijkstra_path(graph: nx.Graph, start: int, goal: int, weight: str = "weight") -> PathResult:
    """Uniform-cost search for the cheapest path between two nodes.

    Parameters
    ----------
    graph:
        Undirected graph whose edges carry the attribute ``weight``.
    start, goal:
        Node ids.
    weight:
        Name of the edge attribute holding the cost.

    Returns
    -------
    PathResult
    """
    t0 = time.perf_counter()
    if start not in graph or goal not in graph:
        return PathResult("dijkstra", False, message="Start or goal is not a graph node")
    adj = graph.adj
    distance: Dict[int, float] = {start: 0.0}
    came_from: Dict[int, int] = {}
    visited: set = set()
    heap: List[Tuple[float, int]] = [(0.0, start)]
    expanded, generated = 0, 1
    while heap:
        d_current, current = heapq.heappop(heap)
        if current in visited:
            continue                                    # stale entry (lazy deletion)
        visited.add(current)
        expanded += 1
        if current == goal:
            return PathResult("dijkstra", True, _reconstruct(came_from, goal), d_current,
                              expanded, generated, time.perf_counter() - t0,
                              message="Optimal path found")
        for neighbour, data in adj[current].items():
            if neighbour in visited:
                continue
            tentative = d_current + data[weight]      # edge relaxation
            if tentative < distance.get(neighbour, math.inf):
                distance[neighbour] = tentative
                came_from[neighbour] = current
                heapq.heappush(heap, (tentative, neighbour))
                generated += 1
    return PathResult("dijkstra", False, expanded=expanded, generated=generated,
                      runtime_s=time.perf_counter() - t0,
                      message="No path: start and goal lie in different components")


def astar_path(graph: nx.Graph, start: int, goal: int, weight: str = "weight",
               heuristic_weight: float = 1.0) -> PathResult:
    """A* search with the great-circle heuristic.

    Parameters
    ----------
    graph:
        Undirected graph; nodes carry ``lat``/``lon`` (degrees), edges the
        attribute ``weight``.
    start, goal:
        Node ids.
    weight:
        Name of the edge attribute holding the cost.
    heuristic_weight:
        1.0 keeps the heuristic admissible; larger values give weighted A*.

    Returns
    -------
    PathResult
    """
    t0 = time.perf_counter()
    if start not in graph or goal not in graph:
        return PathResult("astar", False, message="Start or goal is not a graph node")
    nodes = graph.nodes
    adj = graph.adj
    goal_lat = math.radians(nodes[goal]["lat"])
    goal_lon = math.radians(nodes[goal]["lon"])
    cos_goal = math.cos(goal_lat)
    scale = heuristic_weight * 2.0 * EARTH_RADIUS_KM
    h_cache: Dict[int, float] = {}

    def heuristic(n: int) -> float:
        """Great-circle distance from node ``n`` to the goal (haversine), km."""
        value = h_cache.get(n)
        if value is None:
```

```
            lat = math.radians(nodes[n]["lat"])
            lon = math.radians(nodes[n]["lon"])
            a = (math.sin((lat - goal_lat) * 0.5) ** 2
                 + math.cos(lat) * cos_goal * math.sin((lon - goal_lon) * 0.5) ** 2)
            value = scale * math.asin(math.sqrt(min(max(a, 0.0), 1.0)))
            h_cache[n] = value
        return value

    g_score: Dict[int, float] = {start: 0.0}
    came_from: Dict[int, int] = {}
    closed: set = set()
    heap: List[Tuple[float, float, int]] = [(heuristic(start), 0.0, start)]
    expanded, generated = 0, 1
    while heap:
        _, g_current, current = heapq.heappop(heap)
        if current in closed:
            continue                                  # stale entry (lazy deletion)
        closed.add(current)
        expanded += 1
        if current == goal:
            return PathResult("astar", True, _reconstruct(came_from, goal), g_current,
                              expanded, generated, time.perf_counter() - t0,
                              message="Optimal path found")
        for neighbour, data in adj[current].items():
            if neighbour in closed:
                continue
            tentative = g_current + data[weight]
            if tentative < g_score.get(neighbour, math.inf):
                g_score[neighbour] = tentative
                came_from[neighbour] = current
                heapq.heappush(heap, (tentative + heuristic(neighbour), tentative, neighbour))
                generated += 1
    return PathResult("astar", False, expanded=expanded, generated=generated,
                      runtime_s=time.perf_counter() - t0,
                      message="No path: start and goal lie in different components")


def path_cost(graph: nx.Graph, path: List[int], weight: str = "weight") -> Optional[float]:
    """Sum of edge costs along a node path; ``None`` if an edge is missing."""
    total = 0.0
    for a, b in zip(path[:-1], path[1:]):
        data = graph.adj[a].get(b)
        if data is None:
            return None
        total += data[weight]
    return total
```

5 Module genetic.py

```
"""Genetic algorithm for the ship-course problem on the point graph.

Encoding
--------
A chromosome is a route: the sequence of graph nodes ``[s, v1, ..., vk, t]``
from the departure ``s`` to the destination ``t`` in which every two
consecutive nodes are joined by an edge and no node occurs twice. The length
of the chromosome is variable. This path encoding follows Ahn and
Ramakrishna (2002); every chromosome is a feasible route by construction, so
the fitness never has to judge a broken route.

Fitness
-------
The fitness to be minimised is the route cost of :mod:`.cost`::

    F(P) = L(P) + alpha * L_c(P) + beta * L_d(P)

Operators
---------
* **Initialisation** - every individual is grown by a randomised depth-first
  walk from ``s`` towards ``t``. At each step the next node is drawn among the
  unvisited neighbours with probability proportional to
  ``exp(kappa * g)``, where ``g`` is the progress towards the goal divided by
  the cost of the edge. ``kappa`` is drawn anew for each walk, so the initial
  population mixes almost straight routes with strongly meandering ones. A
  dead end is left by backtracking. In ``via_share`` of the cases the walk is
  first led to a random intermediate node drawn in a circle around the
  midpoint of its end nodes, which spreads the population over different
  corridors (for example inshore and offshore of a cape).
* **Selection** - tournament selection.
* **Crossover** - one-point crossover at a common node: both parents are cut
  at a node they share and the head of one parent is joined to the tail of the
  other. Loops that may appear are removed.
* **Mutation** - one of two moves:
  *re-routing* replaces the part of the route between two random positions by
  a new random walk (possibly through an intermediate node) between the same
  nodes; *shortcut* replaces a sub-route
  ``v_i ... v_j`` by the direct edge ``(v_i, v_j)`` when that edge exists and
  is cheaper.
* **Elitism** - the best individuals pass unchanged to the next generation,
  so the best cost never increases.
* **Termination** - after the given number of generations, after
  ``stagnation_limit`` generations without improvement, or on a stop request
  from the user interface.

All random numbers come from one ``random.Random`` seeded with
``GAConfig.seed``, so a run is reproducible.
"""
from __future__ import annotations

import math
import random
import statistics
import time
from dataclasses import asdict, dataclass
from typing import Callable, Dict, List, Optional, Sequence, Tuple

import networkx as nx
import numpy as np
from scipy.spatial import cKDTree

from ..config import EARTH_RADIUS_KM
from .search import PathResult

Route = List[int]
#: ``progress(generation, best_cost, median_cost)``.
GAProgress = Optional[Callable[[int, float, float], None]]
#: Returns True when the user asked to stop the run.
StopFlag = Optional[Callable[[], bool]]


@dataclass(frozen=True)
class GAConfig:
    """Parameters of the genetic algorithm (all editable in the GUI).

    Attributes
    ----------
    population_size:
        Number of routes in a generation.
    generations:
        Maximal number of generations.
    crossover_rate:
```

```
        Probability that two selected parents are recombined.
    mutation_rate:
        Probability that a child is mutated.
    tournament_size:
        Number of individuals compared in one tournament.
    elite:
        Number of best individuals copied unchanged.
    stagnation_limit:
        Stop after this many generations without improvement (0 = never).
    shortcut_share:
        Share of mutations that are shortcuts (the rest are re-routings).
    reroute_window:
        Largest number of route positions replaced by one re-routing.
    kappa_range:
        Range of the goal-attraction coefficient of the random walk.
    via_share:
        Share of random walks (initial routes and re-routings) that are led
        through a random intermediate node.
    via_spread:
        Range of the radius of the circle in which the intermediate node is
        drawn, as a fraction of the distance between the two end nodes.
    seed:
        Seed of the random generator.
    """

    population_size: int = 100
    generations: int = 300
    crossover_rate: float = 0.8
    mutation_rate: float = 0.3
    tournament_size: int = 3
    elite: int = 2
    stagnation_limit: int = 80
    shortcut_share: float = 0.4
    reroute_window: int = 60
    kappa_range: Tuple[float, float] = (1.0, 15.0)
    via_share: float = 0.5
    via_spread: Tuple[float, float] = (0.1, 0.5)
    seed: int = 42

    def to_dict(self) -> Dict:
        return asdict(self)


class _RouteSpace:
    """Adjacency, costs and geometry of the graph in plain Python containers.

    NetworkX attribute look-ups are convenient but slow inside the inner loop
    of the algorithm, so the graph is copied once into dictionaries.
    """

    def __init__(self, graph: nx.Graph, weight: str = "weight") -> None:
        self.cost: Dict[int, Dict[int, float]] = {}
        self.length: Dict[int, Dict[int, float]] = {}
        for u, nbrs in graph.adj.items():
            self.cost[u] = {v: d[weight] for v, d in nbrs.items()}
            self.length[u] = {v: d["length"] for v, d in nbrs.items()}
        self.lat = {n: math.radians(d["lat"]) for n, d in graph.nodes(data=True)}
        self.lon = {n: math.radians(d["lon"]) for n, d in graph.nodes(data=True)}
        # Spatial index of the nodes, used to draw random intermediate nodes.
        self.nodes = list(graph.nodes)
        lat = np.radians([graph.nodes[n]["lat"] for n in self.nodes])
        lon = np.radians([graph.nodes[n]["lon"] for n in self.nodes])
        self.xyz = EARTH_RADIUS_KM * np.column_stack(
            (np.cos(lat) * np.cos(lon), np.cos(lat) * np.sin(lon), np.sin(lat)))
        self.position = {n: i for i, n in enumerate(self.nodes)}
        self.tree = cKDTree(self.xyz)
        self.evaluations = 0

    def distance(self, a: int, b: int) -> float:
        """Great-circle distance between two nodes, km (haversine)."""
        la, lb = self.lat[a], self.lat[b]
        h = (math.sin((lb - la) * 0.5) ** 2
             + math.cos(la) * math.cos(lb) * math.sin((self.lon[b] - self.lon[a]) * 0.5) ** 2)
        return 2.0 * EARTH_RADIUS_KM * math.asin(math.sqrt(min(max(h, 0.0), 1.0)))

    def route_cost(self, route: Sequence[int]) -> float:
        """Fitness ``F`` of a route (sum of edge costs)."""
        self.evaluations += 1
        cost = self.cost
        return sum(cost[a][b] for a, b in zip(route[:-1], route[1:]))

    def walk(self, src: int, dst: int, rng: random.Random, kappa: float,
             forbidden: Optional[set] = None, max_steps: Optional[int] = None) -> Optional[Route]:
        """Randomised depth-first walk from ``src`` to ``dst``.
```

```
    Parameters
    ----------
    src, dst:
        End nodes.
    rng:
        Random generator.
    kappa:
        Goal attraction; 0 gives a uniform random walk.
    forbidden:
        Nodes the walk must not enter (the rest of the route).
    max_steps:
        Upper bound on the number of forward steps (``None`` = unbounded;
        the walk then ends at the latest when the component is exhausted).

    Returns
    -------
    list of int or None
        A simple path ``[src, ..., dst]`` or ``None`` if none was found.
    """
    visited = set(forbidden or ())
    visited.discard(dst)
    visited.add(src)
    stack = [src]
    h_cache: Dict[int, float] = {}

    def h(n: int) -> float:
        value = h_cache.get(n)
        if value is None:
            value = self.distance(n, dst)
            h_cache[n] = value
        return value

    steps = 0
    while stack:
        u = stack[-1]
        if u == dst:
            return stack
        if max_steps is not None and steps >= max_steps:
            return None
        options = [v for v in self.cost[u] if v not in visited]
        if not options:
            stack.pop()                          # dead end: backtrack
            continue
        if dst in options:
            v = dst
        else:
            hu = h(u)
            weights = []
            for v in options:
                edge = self.cost[u][v] or 1e-9
                gain = (hu - h(v)) / edge        # progress per unit of cost
                weights.append(math.exp(kappa * gain))
            v = rng.choices(options, weights=weights, k=1)[0]
        visited.add(v)
        stack.append(v)
        steps += 1
    return None


def random_via(self, a: int, b: int, rng: random.Random, spread: float,
               forbidden: set) -> Optional[int]:
    """Random node inside a circle around the midpoint of ``a`` and ``b``.

    The radius is ``spread`` times the distance between the two nodes
    (at least 5 km), so the drawn node can pull a route far to the side
    of the direct line.
    """
    pa, pb = self.xyz[self.position[a]], self.xyz[self.position[b]]
    radius = max(spread * float(np.linalg.norm(pb - pa)), 5.0)
    candidates = self.tree.query_ball_point(0.5 * (pa + pb), radius)
    candidates = [self.nodes[i] for i in candidates]
    candidates = [n for n in candidates if n not in forbidden and n != a and n != b]
    if not candidates:
        return None
    return rng.choice(candidates)

def via_walk(self, src: int, dst: int, rng: random.Random, kappa: float, spread: float,
             forbidden: Optional[set] = None, max_steps: Optional[int] = None) -> Optional[Route]:
    """Walk ``src -> via -> dst`` through a random intermediate node."""
    forbidden = set(forbidden or ())
    via = self.random_via(src, dst, rng, spread, forbidden)
    if via is None:
        return self.walk(src, dst, rng, kappa, forbidden, max_steps)
    first = self.walk(src, via, rng, kappa, forbidden | {dst}, max_steps)
    if first is None:
```

```
            return None
        second = self.walk(via, dst, rng, kappa, forbidden | set(first[:-1]), max_steps)
        if second is None:
            return None
        return first + second[1:]


def _remove_loops(route: Route) -> Route:
    """Cut out every cycle so that each node occurs at most once."""
    out: Route = []
    position: Dict[int, int] = {}
    for node in route:
        if node in position:
            cut = position[node]
            for removed in out[cut + 1:]:
                del position[removed]
            del out[cut + 1:]
        else:
            position[node] = len(out)
            out.append(node)
    return out


def genetic_path(graph: nx.Graph, start: int, goal: int, config: GAConfig = GAConfig(),
                 weight: str = "weight", progress: GAProgress = None,
                 should_stop: StopFlag = None) -> PathResult:
    """Search a low-cost route with the genetic algorithm.

    Parameters
    ----------
    graph:
        Navigation graph with edge attributes ``weight`` and ``length`` and
        node attributes ``lat``/``lon``.
    start, goal:
        Node ids of the departure and the destination.
    config:
        Parameters of the algorithm.
    weight:
        Edge attribute holding the cost.
    progress:
        Optional callback ``progress(generation, best, median)``.
    should_stop:
        Optional callback; when it returns True the run ends after the
        current generation.

    Returns
    -------
    PathResult
        ``expanded`` = number of fitness evaluations, ``generated`` = number
        of generations run, ``best_history``/``median_history`` = convergence
        curves (index 0 = initial population).
    """
    t0 = time.perf_counter()
    if start not in graph or goal not in graph:
        return PathResult("genetic", False, message="Start or goal is not a graph node")
    if start == goal:
        return PathResult("genetic", True, [start], 0.0, runtime_s=time.perf_counter() - t0,
                          message="Start and goal coincide")
    if not nx.has_path(graph, start, goal):
        return PathResult("genetic", False, runtime_s=time.perf_counter() - t0,
                          message="No path: start and goal lie in different components")

    rng = random.Random(config.seed)
    space = _RouteSpace(graph, weight)
    k_lo, k_hi = config.kappa_range
    s_lo, s_hi = config.via_spread

    def random_route(src: int, dst: int, forbidden: Optional[set] = None,
                     max_steps: Optional[int] = None) -> Optional[Route]:
        """Random walk, led through an intermediate node in ``via_share`` of cases."""
        kappa = rng.uniform(k_lo, k_hi)
        if rng.random() < config.via_share:
            return space.via_walk(src, dst, rng, kappa, rng.uniform(s_lo, s_hi),
                                  forbidden, max_steps)
        return space.walk(src, dst, rng, kappa, forbidden, max_steps)

    # ---- initial population ------------------------------------------------
    population: List[Route] = []
    attempts = 0
    while len(population) < config.population_size and attempts < 5 * config.population_size:
        attempts += 1
        route = random_route(start, goal)
        if route is not None:
            population.append(route)
    if not population:
```

```
    return PathResult("genetic", False, runtime_s=time.perf_counter() - t0,
                      message="Could not create an initial population")
while len(population) < config.population_size:
    population.append(list(rng.choice(population)))
fitness = [space.route_cost(r) for r in population]

# ---- operators ---------------------------------------------------------
def tournament() -> Route:
    contenders = rng.sample(range(len(population)), min(config.tournament_size, len(population)))
    return population[min(contenders, key=lambda i: fitness[i])]

def crossover(a: Route, b: Route) -> Tuple[Route, Route]:
    common = list(set(a[1:-1]) & set(b[1:-1]))
    if not common:
        return list(a), list(b)
    node = rng.choice(common)
    i, j = a.index(node), b.index(node)
    return _remove_loops(a[:i] + b[j:]), _remove_loops(b[:j] + a[i:])

def reroute(route: Route) -> Route:
    if len(route) < 3:
        return route
    span = rng.randint(2, max(2, min(config.reroute_window, len(route) - 1)))
    i = rng.randrange(0, len(route) - span)
    j = i + span
    forbidden = set(route[:i]) | set(route[j + 1:])
    sub = random_route(route[i], route[j], forbidden, max_steps=8 * span + 40)
    if sub is None:
        return route
    return route[:i] + sub + route[j + 1:]

def shortcut(route: Route) -> Route:
    if len(route) < 3:
        return route
    i = rng.randrange(0, len(route) - 2)
    adj = space.cost[route[i]]
    for j in range(min(len(route) - 1, i + config.reroute_window), i + 1, -1):
        if route[j] in adj:
            old = sum(space.cost[a][b] for a, b in zip(route[i:j], route[i + 1:j + 1]))
            if adj[route[j]] < old:
                return route[:i + 1] + route[j:]
            break
    return route

def mutate(route: Route) -> Route:
    if rng.random() < config.shortcut_share:
        return shortcut(route)
    return reroute(route)

# ---- evolution ---------------------------------------------------------
best_i = min(range(len(population)), key=lambda i: fitness[i])
best, best_fit = list(population[best_i]), fitness[best_i]
best_history = [best_fit]
median_history = [statistics.median(fitness)]
stagnant = 0
generation = 0
stopped = False
for generation in range(1, config.generations + 1):
    order = sorted(range(len(population)), key=lambda i: fitness[i])
    offspring: List[Route] = [list(population[i]) for i in order[:config.elite]]
    while len(offspring) < config.population_size:
        a, b = tournament(), tournament()
        if rng.random() < config.crossover_rate:
            children = crossover(a, b)
        else:
            children = (list(a), list(b))
        for child in children:
            if rng.random() < config.mutation_rate:
                child = mutate(child)
            offspring.append(child)
            if len(offspring) == config.population_size:
                break
    population = offspring
    fitness = [space.route_cost(r) for r in population]
    gen_best = min(range(len(population)), key=lambda i: fitness[i])
    if fitness[gen_best] < best_fit - 1e-9:
        best, best_fit = list(population[gen_best]), fitness[gen_best]
        stagnant = 0
    else:
        stagnant += 1
    best_history.append(best_fit)
    median_history.append(statistics.median(fitness))
    if progress is not None:
        progress(generation, best_fit, median_history[-1])
    if should_stop is not None and should_stop():
```

```
            stopped = True
            break
        if config.stagnation_limit and stagnant >= config.stagnation_limit:
            break

    reason = ("stopped by the user" if stopped else
              "no improvement" if config.stagnation_limit and stagnant >= config.stagnation_limit
              else "generation limit")
    return PathResult(
        algorithm="genetic", found=True, path=best, cost=best_fit,
        expanded=space.evaluations, generated=generation,
        runtime_s=time.perf_counter() - t0,
        best_history=best_history, median_history=median_history,
        message=f"Best route after {generation} generations ({reason}), {len(best)} nodes",
    )
```